

# Transformers

# Types of Sequential Data

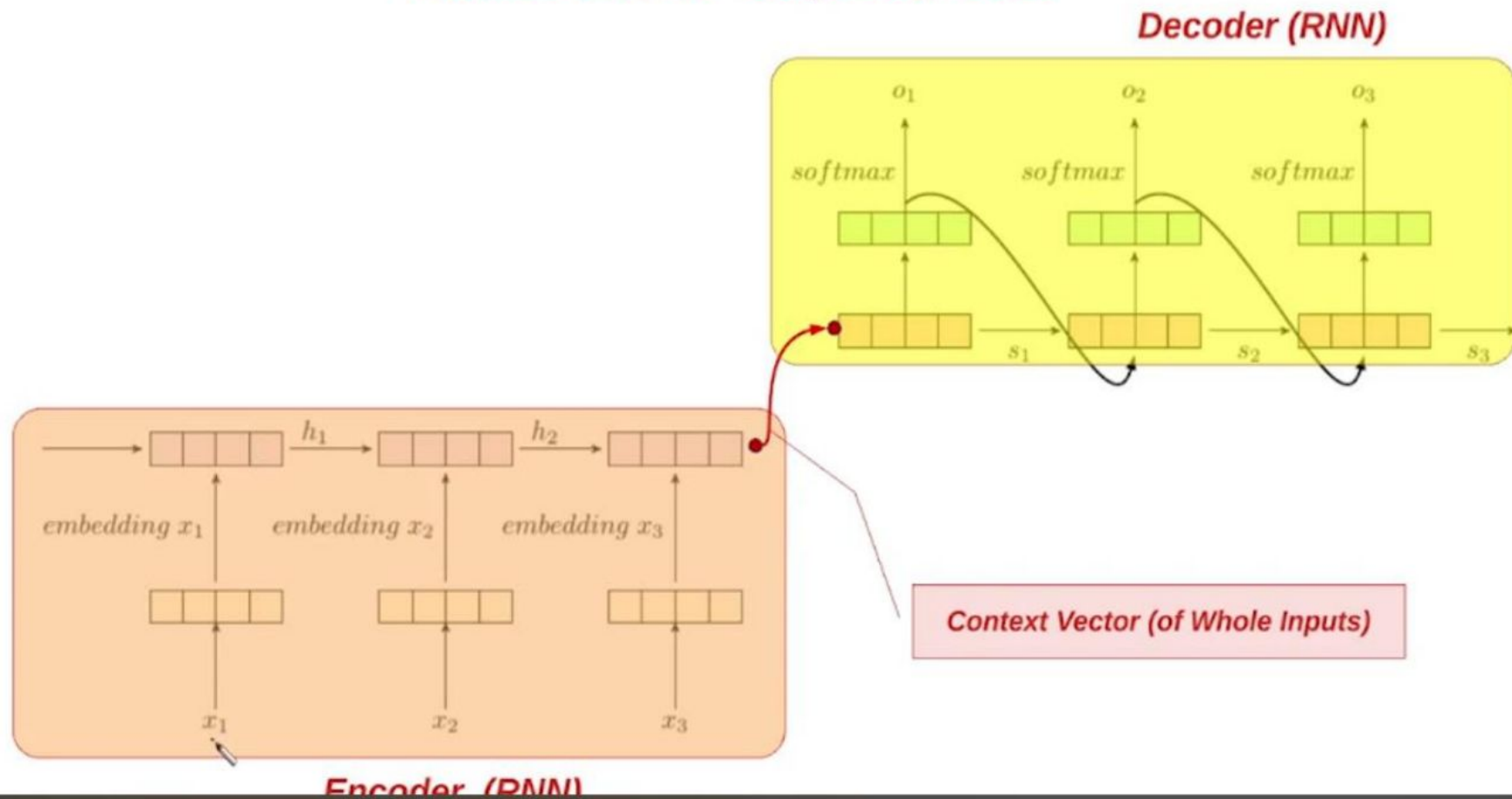
- **Text Data:** Sentences or documents where each word or character is a sequential element.
- **Time Series Data:** Data points collected or recorded at specific time intervals, such as stock prices, weather data, or sensor readings.
- **Speech/Audio Data:** Sequences of sound waves, often processed as frames or segments.
- **Genomic Data:** DNA sequences where the order of nucleotides is crucial.

# Models for Sequential Data

## **Recurrent Neural Networks (RNNs):**

- RNNs are designed to handle sequential data by maintaining a hidden state that captures information from previous time steps.
- Variants like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) address issues like vanishing gradients, allowing the model to capture long-range dependencies.

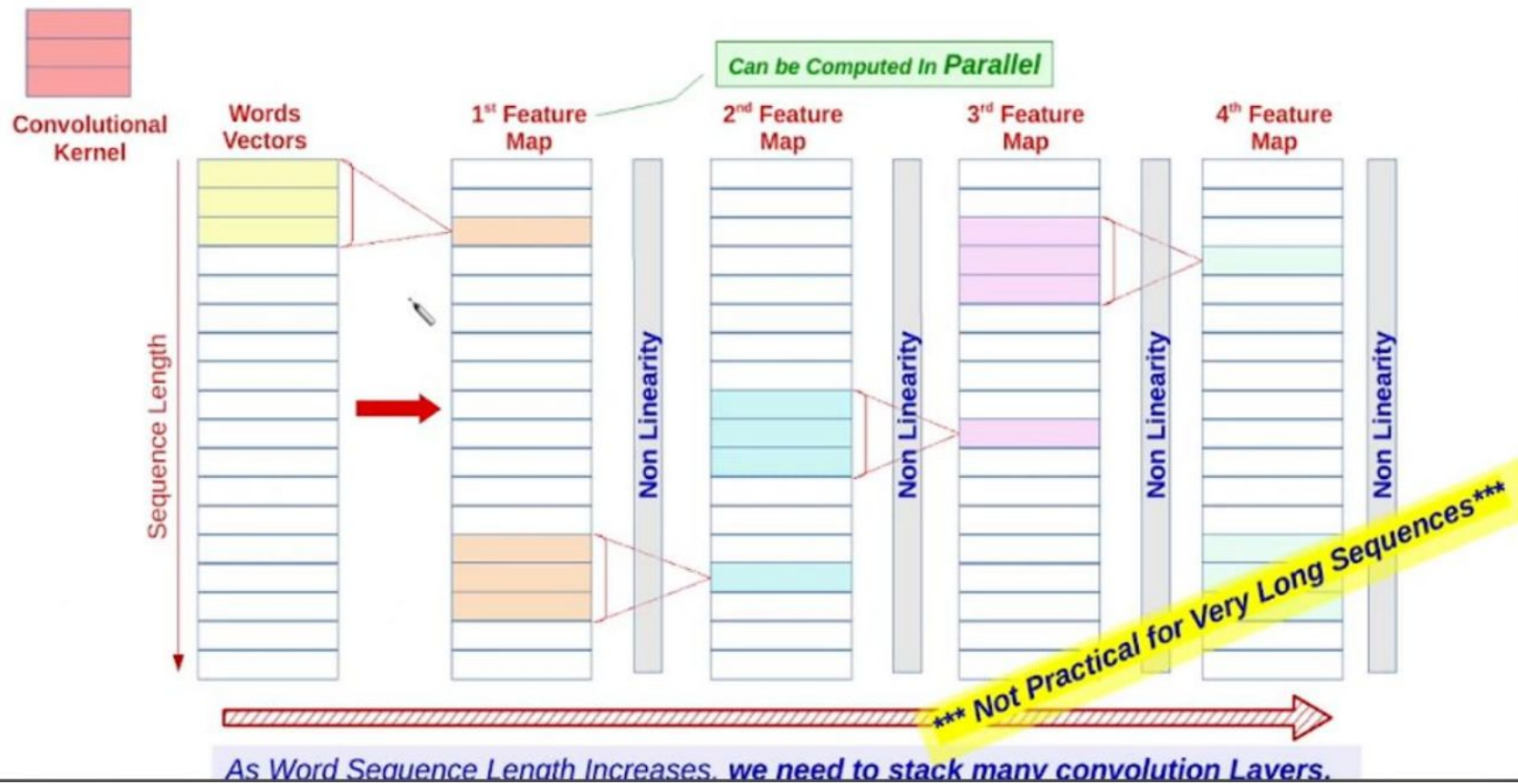
## Traditional RNN Seq2Seq model



## **Convolutional Neural Networks (CNNs):**

- Though traditionally used for image data, CNNs can also be applied to sequential data. For example, 1D convolutions can be used to capture local patterns in time series or text data.
- Temporal Convolutional Networks (TCNs) extend CNNs to capture longer-range dependencies in sequences.

# 1-D Convolutional Neural Network



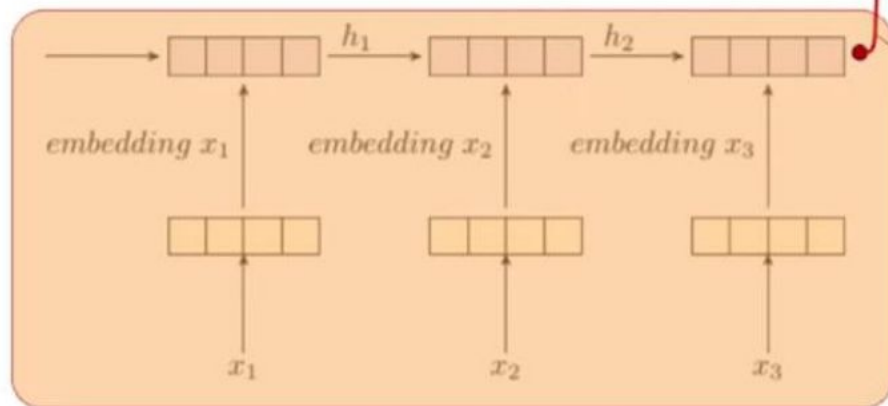
## **Sequence-to-Sequence (Seq2Seq) Models:**

- Seq2Seq models, often using RNNs or Transformers, are designed for tasks where the output sequence length can differ from the input sequence length, such as translation or summarization.

## Traditional RNN Seq2Seq model (Challenges)

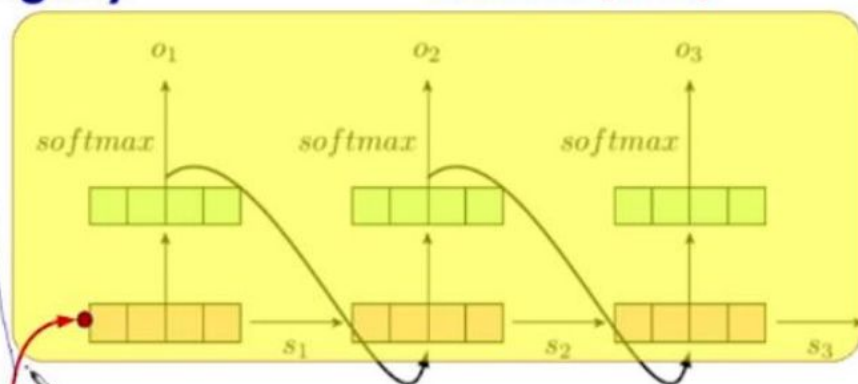
### [1] Bottleneck

The meaning of the entire input sequence is Expected to be captured by a single context vector with fixed dimensionality



Encoder (RNN)

### Decoder (RNN)



Context Vector (of Whole Inputs)



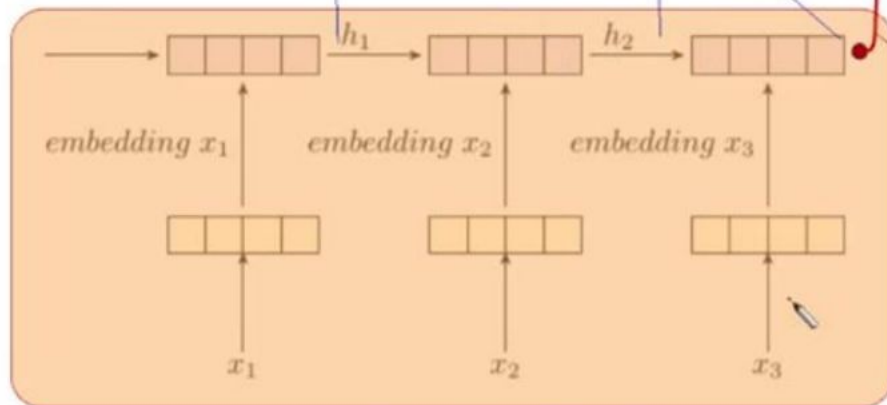
## Traditional RNN Seq2Seq model (Challenges)

[7]

**[2] Sequential Processing**  
Can NOT do processing in Parallel

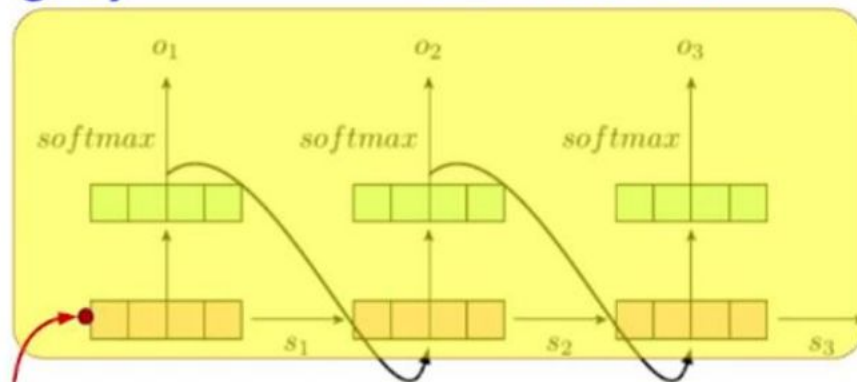
Wait  
Till Processing  $x_1$

Wait  
Till Processing  $x_2$



Encoder (RNN)

Decoder (RNN)



Context Vector (of Whole Inputs)

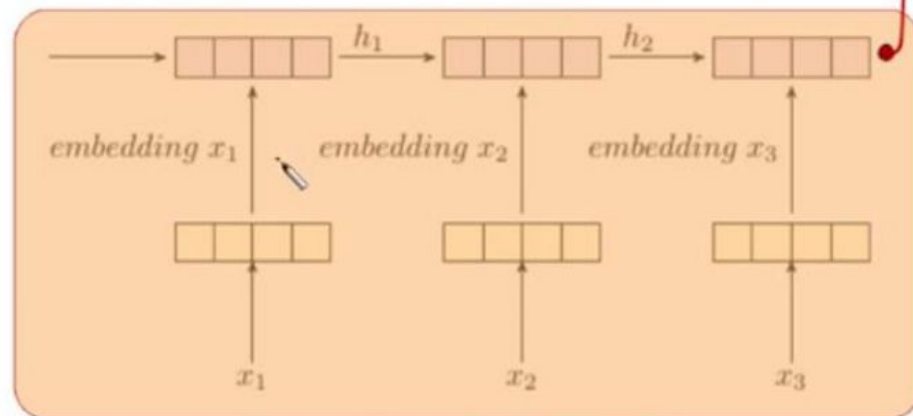
# Traditional RNN Seq2Seq model (Challenges)

[3] Very Long Sequence  
Vanishing Gradient

Short Sequence: **RNN**

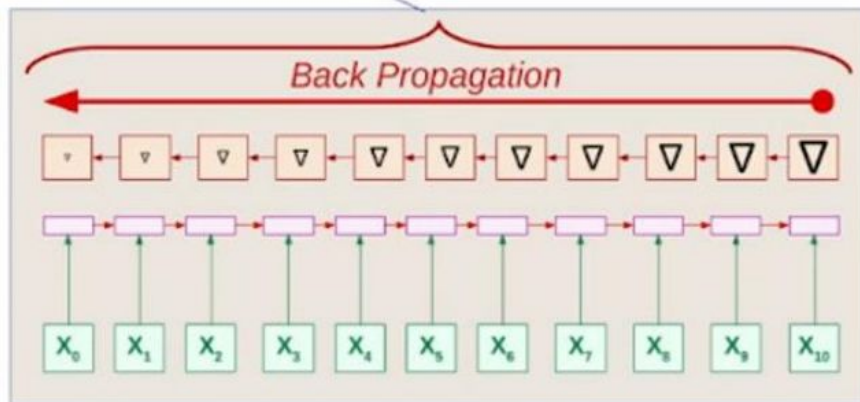
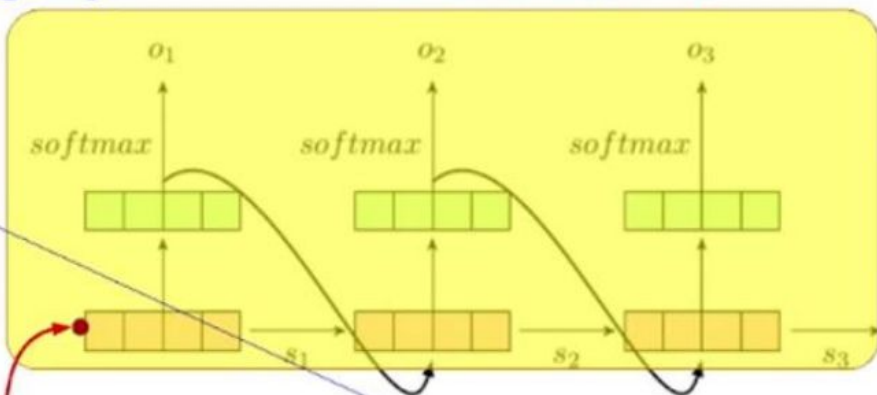
Long Sequence: **LSTM, GRU**

Very Long Sequence: **FAIL**



**Encoder (RNN)**

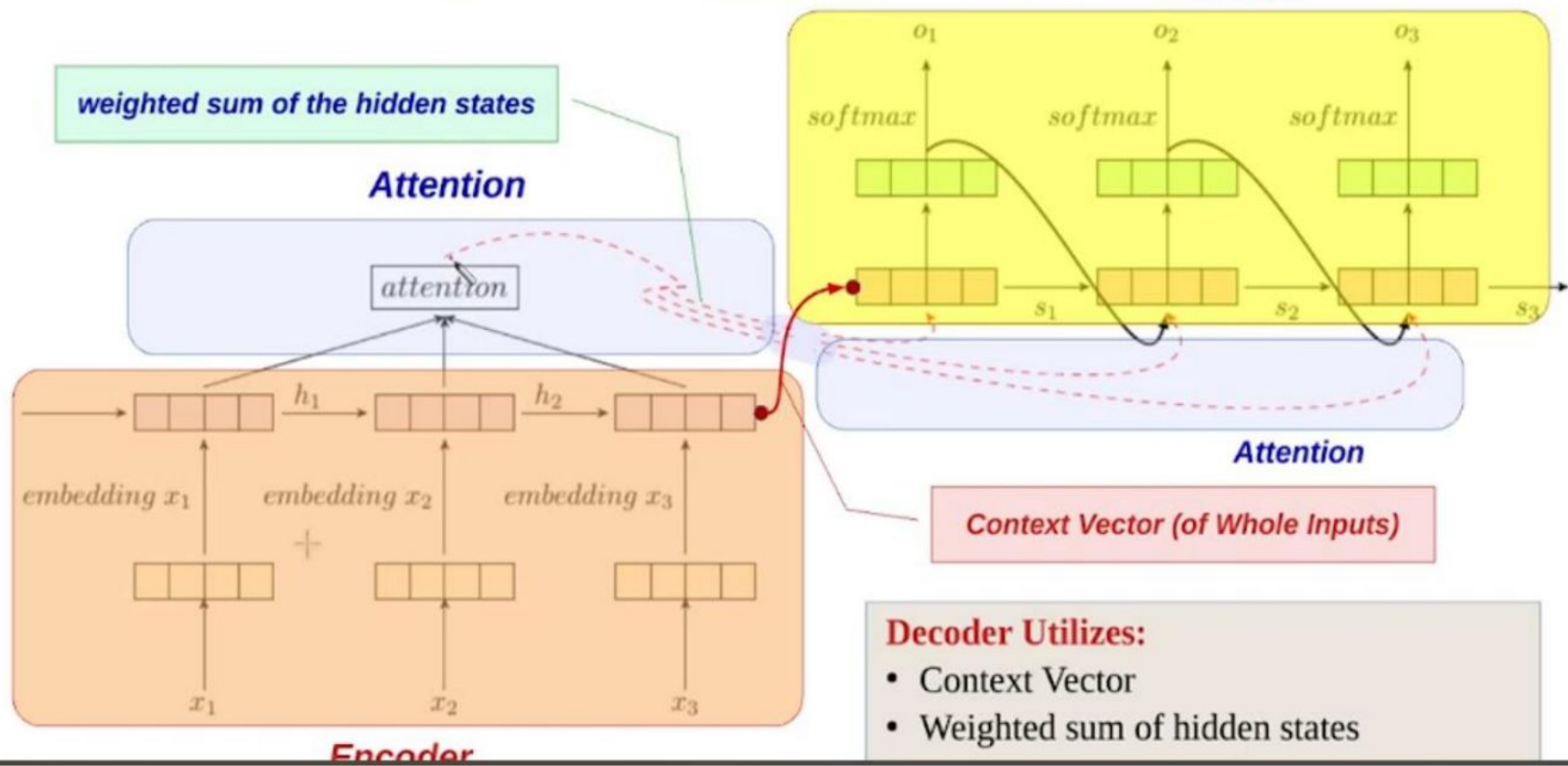
**Decoder (RNN)**



# RNN Seq2Seq model [with Attention]

[To overcome Bottleneck challenge]

*Decoder*



# Types of attention :

Attention mechanisms are a key component in many neural network architectures, particularly in models for natural language processing, computer vision, and other sequential tasks.

Attention allows models to focus on specific parts of the input data when making decisions, which improves the model's ability to capture relevant information.

## 1. Global Attention

- **Definition:** In global attention, the model attends to all positions in the input sequence when generating each output. This means that every part of the input sequence is considered when calculating the attention scores.
- **Usage:** This type of attention is common in machine translation tasks where every word in the source sentence might be relevant to the translation of each word in the target sentence.
- **Pros:** Provides a comprehensive view of the entire input sequence, capturing broad context.
- **Cons:** Computationally expensive, especially with long sequences.

At every output step:

- The model “looks back” at all input words.
- But gives more weight to the most relevant ones (via attention scores).

So it's *global* because it uses the entire input sequence, not just one part.

## 2. Local Attention

- **Definition:** Local attention focuses on a small, fixed-size window of the input sequence around a certain position. This reduces the computational complexity by limiting the attention to a subset of the input.
- **Usage:** Often used in speech recognition or tasks where nearby context is more important than distant parts of the sequence.
- **Pros:** Less computationally intensive than global attention.
- **Cons:** May miss important context outside the fixed window.

### 3. Self-Attention

- **Definition:** Self-attention (or intra-attention) allows a sequence to attend to different positions within itself. This means that each position in the input sequence can focus on other positions in the same sequence when calculating its representation.
- **Usage:** Widely used in Transformer models (BERT, GPT) for tasks like machine translation, text summarization, and language modeling.
- **Pros:** Captures relationships between all parts of the sequence, regardless of distance. Highly parallelizable, which improves computational efficiency.
- **Cons:** Can be computationally expensive for very long sequences due to the need to compute attention scores for every pair of positions.



“The cat sat on the mat.”

When we calculate the representation for “cat”,  
the model looks at:

“The” → helps know it’s a specific cat

“sat” → tells what the cat did

“mat” → gives more context about the scene

## Before self-attention (basic embeddings)

Initially, every word has a fixed embedding (like in Word2Vec or GloVe).

For example, “cat” always has the same vector no matter where it appears.

But that’s a problem the meaning of a word can change with context.

Example:

- “The cat sat on the mat.” → animal
- “He used a cat to lift the car.” → machine (short for *caterpillar jack*)

A single fixed vector can’t capture both meanings.

## After self-attention (contextual embeddings)

Self-attention solves this!

Now, the representation of “cat” is *context-dependent*:

- It looks at other words like “sat” and “mat” → understands this is the animal sense.
- If the sentence were different, its representation would change accordingly.

So the final representation of “cat” = “what the model understands about this word *in this specific context*.”

## 4. Multi-Head Attention

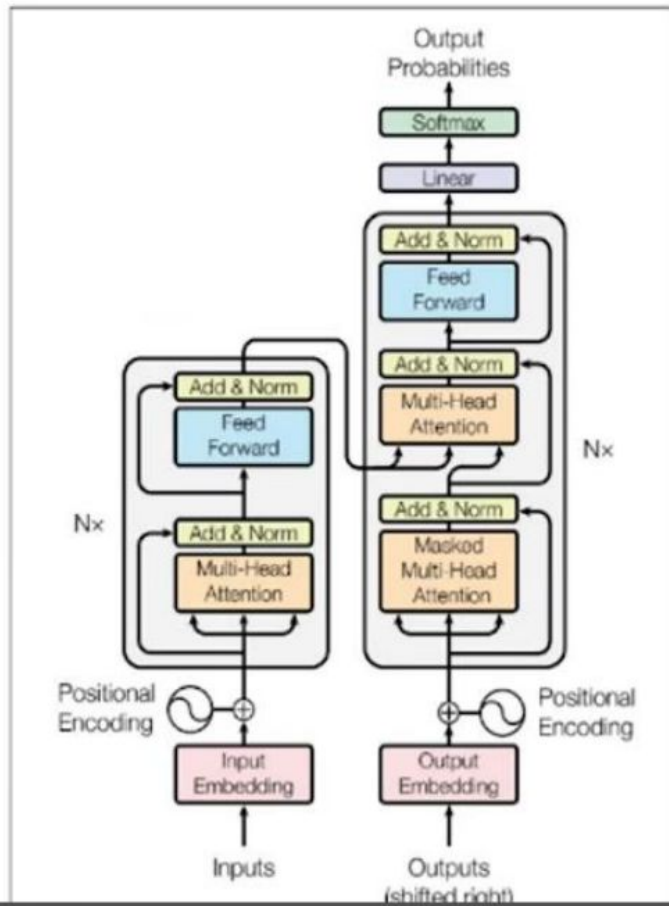
- **Definition:** Multi-head attention involves using multiple attention mechanisms (or "heads") in parallel, each with different learned projections of the input. The results from each head are concatenated and projected to form the final output.
- **Usage:** Common in Transformer models where each head can capture different types of relationships in the data.
- **Pros:** Allows the model to capture different types of relationships in the data simultaneously, improving the representation power.
- **Cons:** Increases the number of parameters and computational complexity.

| Head   | What it might focus on                      |
|--------|---------------------------------------------|
| Head 1 | Grammar relationships (e.g., "cat" → "sat") |
| Head 2 | Semantic meaning (e.g., "cat" → "mat")      |
| Head 3 | Determiners (e.g., "cat" → "The")           |
| Head 4 | Position information                        |

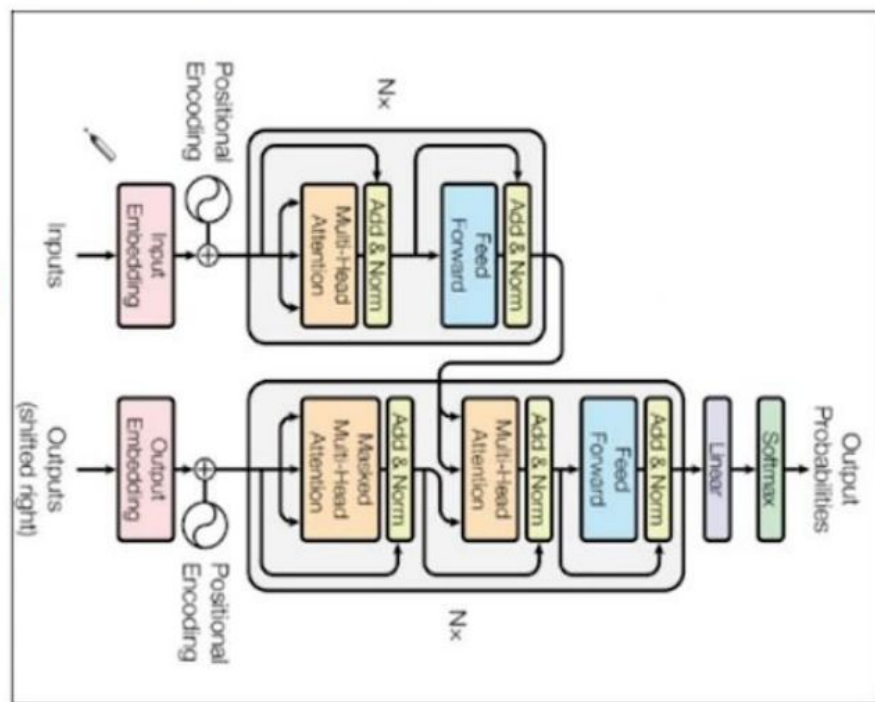
Each head learns different perspectives on the same sentence.

---

# Encoder – Decoder Transformer

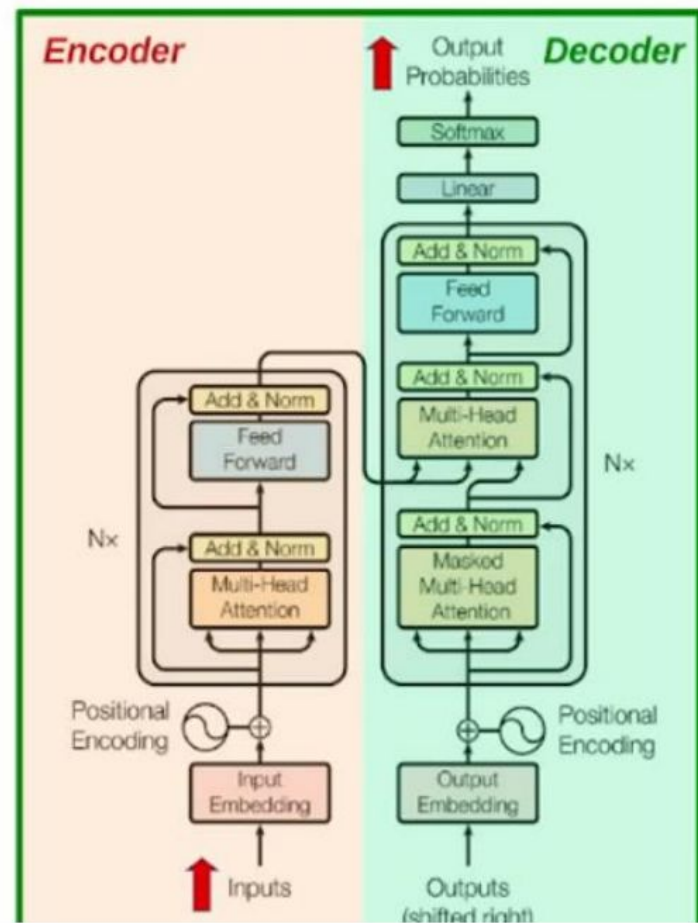
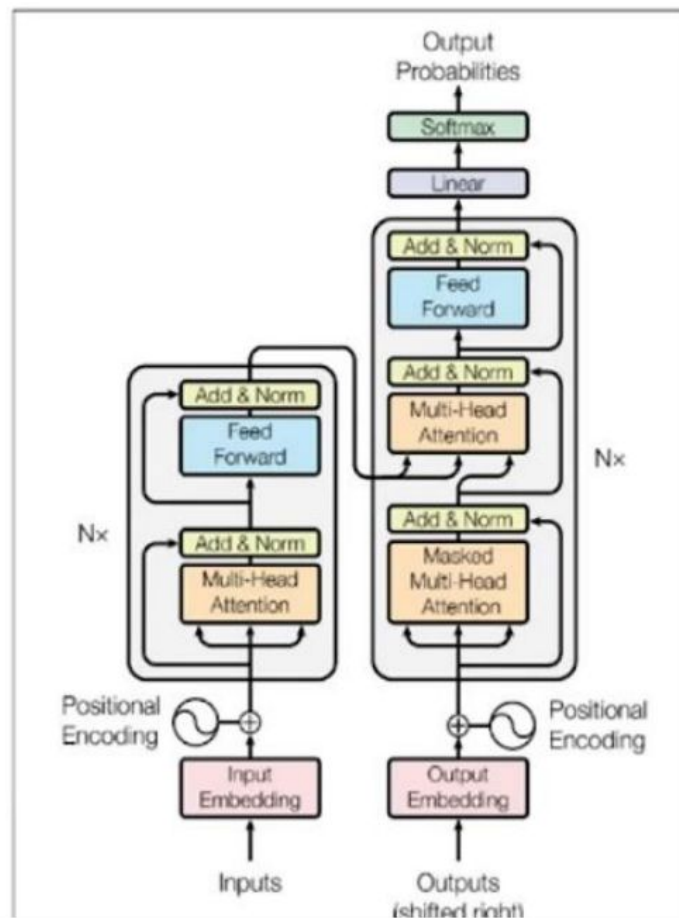


Orientation is not the Issue

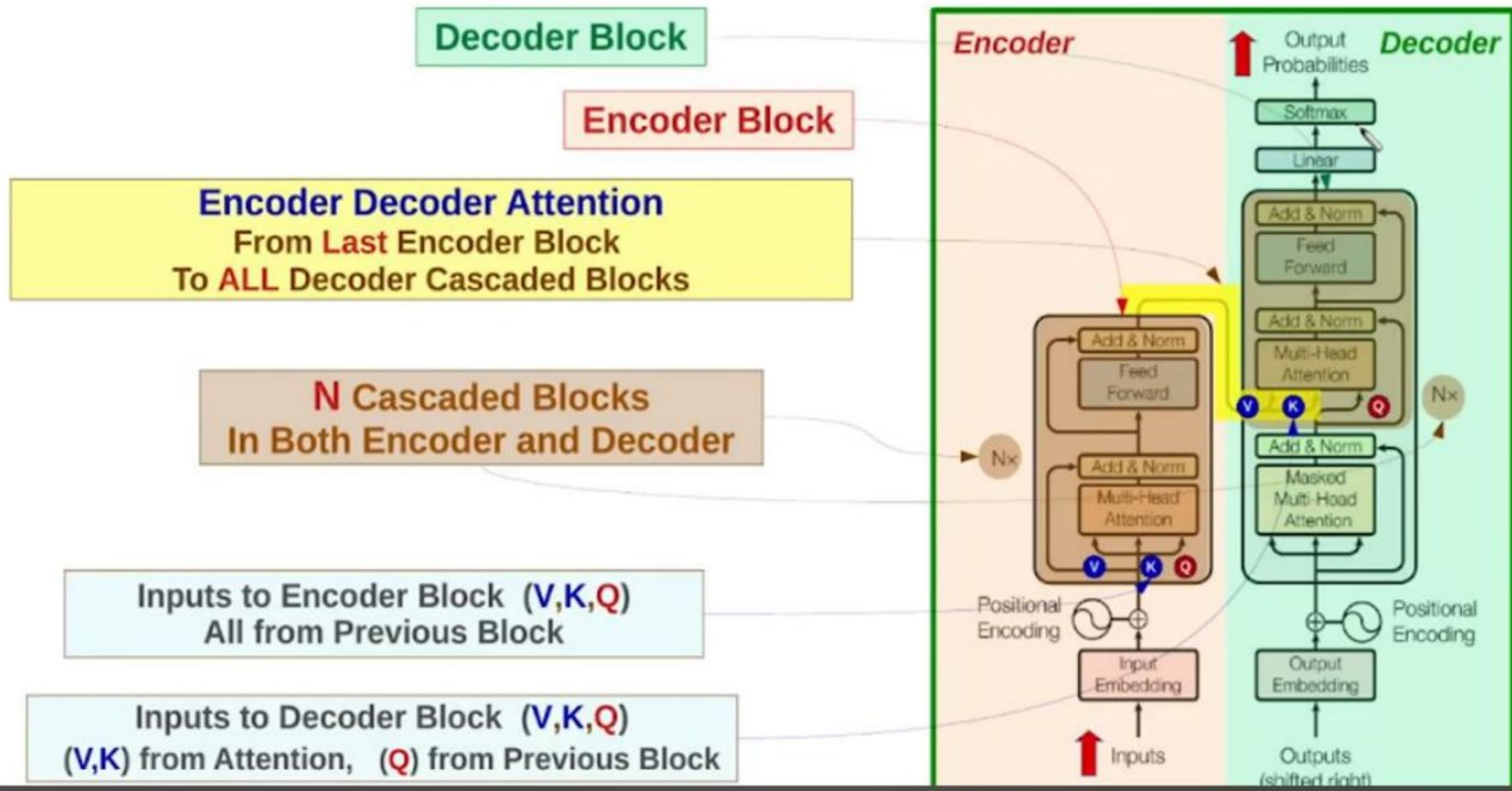


Let us focus on the Building blocks

# Encoder – Decoder Transformer

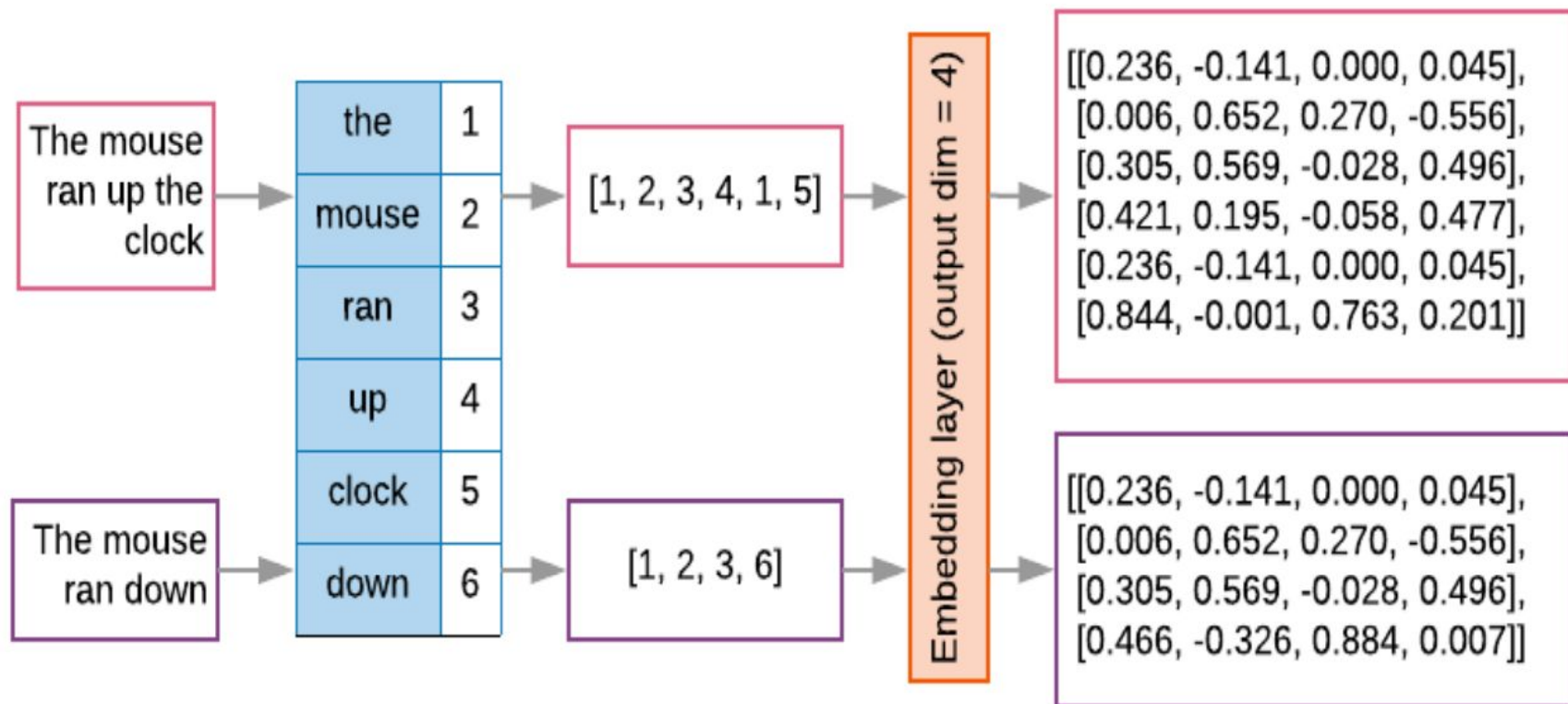


# Encoder – Decoder Transformer





# Token and embedding



## Positional encoding

The cat sat on the mat.”

“The mat sat on the cat.”

Both sentences have the *same words*, but the order changes the meaning.

The Transformer must know which word comes first, second, third.

- Positional Encoding (PE) comes in it adds *positional information* to each word's embedding.

In the Transformer paper (Vaswani et al., 2017), the positional encoding formula is:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

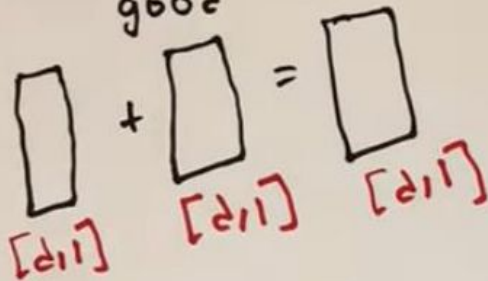
$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

where:

- $\text{pos}$  = position of the word in the sequence (e.g., 1, 2, 3, ...)
- $i$  = dimension index (0, 1, 2, ...)
- $d_{\text{model}}$  = total embedding dimension (512 in the real Transformer, but we used 4 in our simple example)

$d=4$

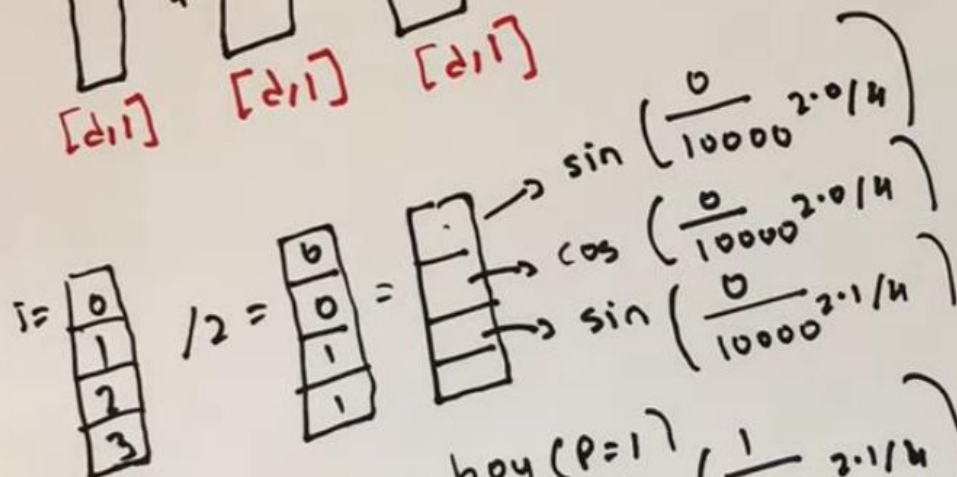
good



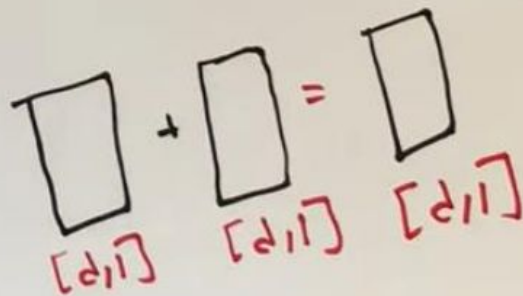
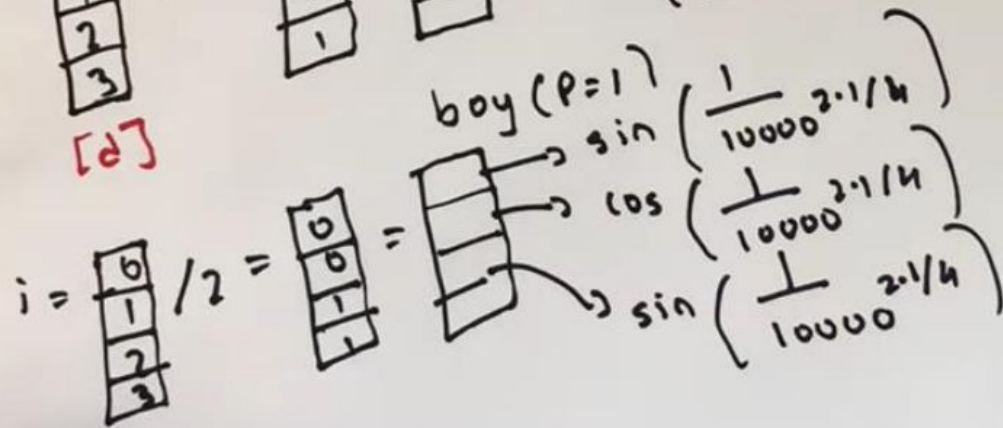
good  $p=0$  boy  $p=1$

$$p \in (p, 2i) = \sin\left(\frac{p}{10000} 2i/d\right)$$

$$p \in (p, 2i+1) = \cos\left(\frac{p}{10000} 2i/d\right)$$



boy ( $p=1$ )



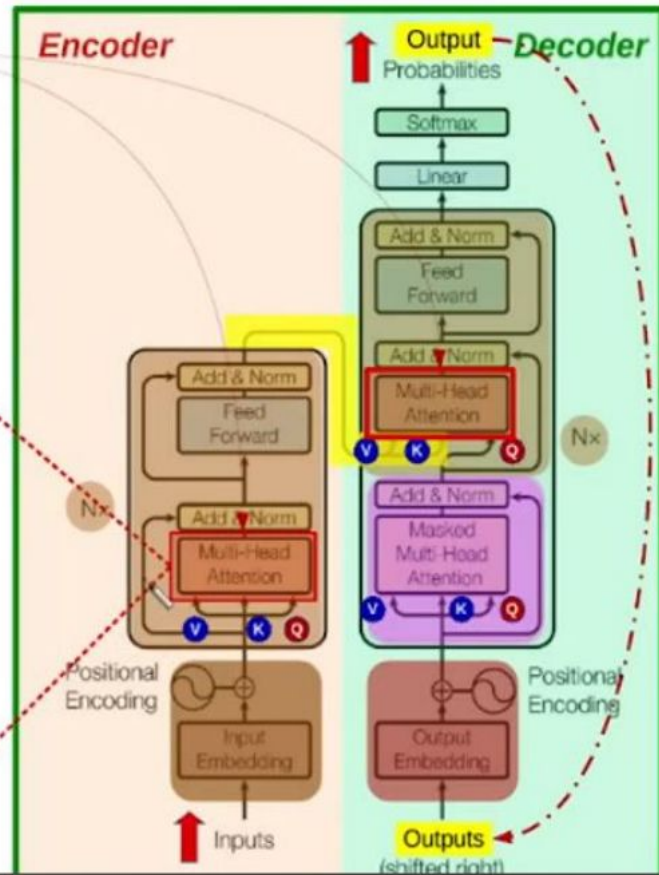
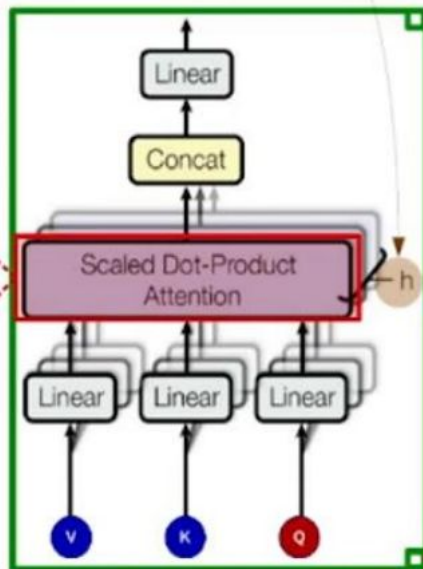
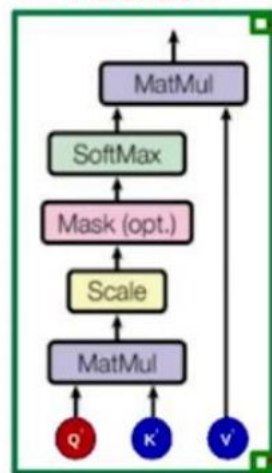
# Encoder – Decoder Transformer

Both Encoder and Decoder Blocks Contain  
“Multi-Head Self Attention” block

“h” Heads  
Outputs of Heads are concatenated

Multi-Head Attention

Scaled Dot-Product  
Attention



# Multi-Head Self-Attention Mechanism

## Key, Query, and Value Vectors

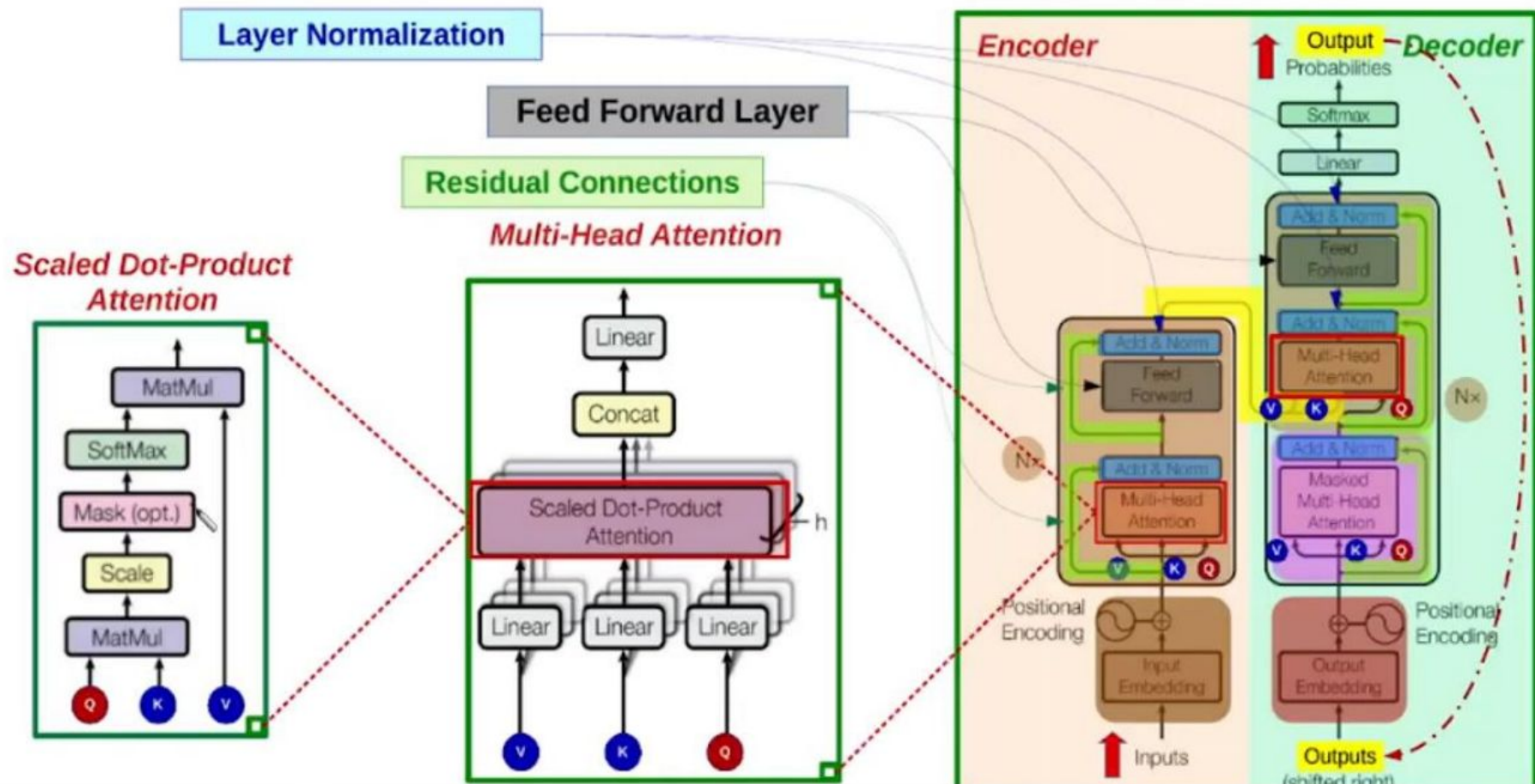
### Create Q, K, V (Query, Key, Value)

For every input vector (each word embedding):

- Multiply it by **three different weight matrices**  
→ to get **Query (Q)**, **Key (K)**, and **Value (V)**.
- **Q (Query)**: What this word is *looking for*
- **K (Key)**: What this word *offers* (its features)
- **V (Value)**: The *information* we actually pass on



# Encoder – Decoder Transformer



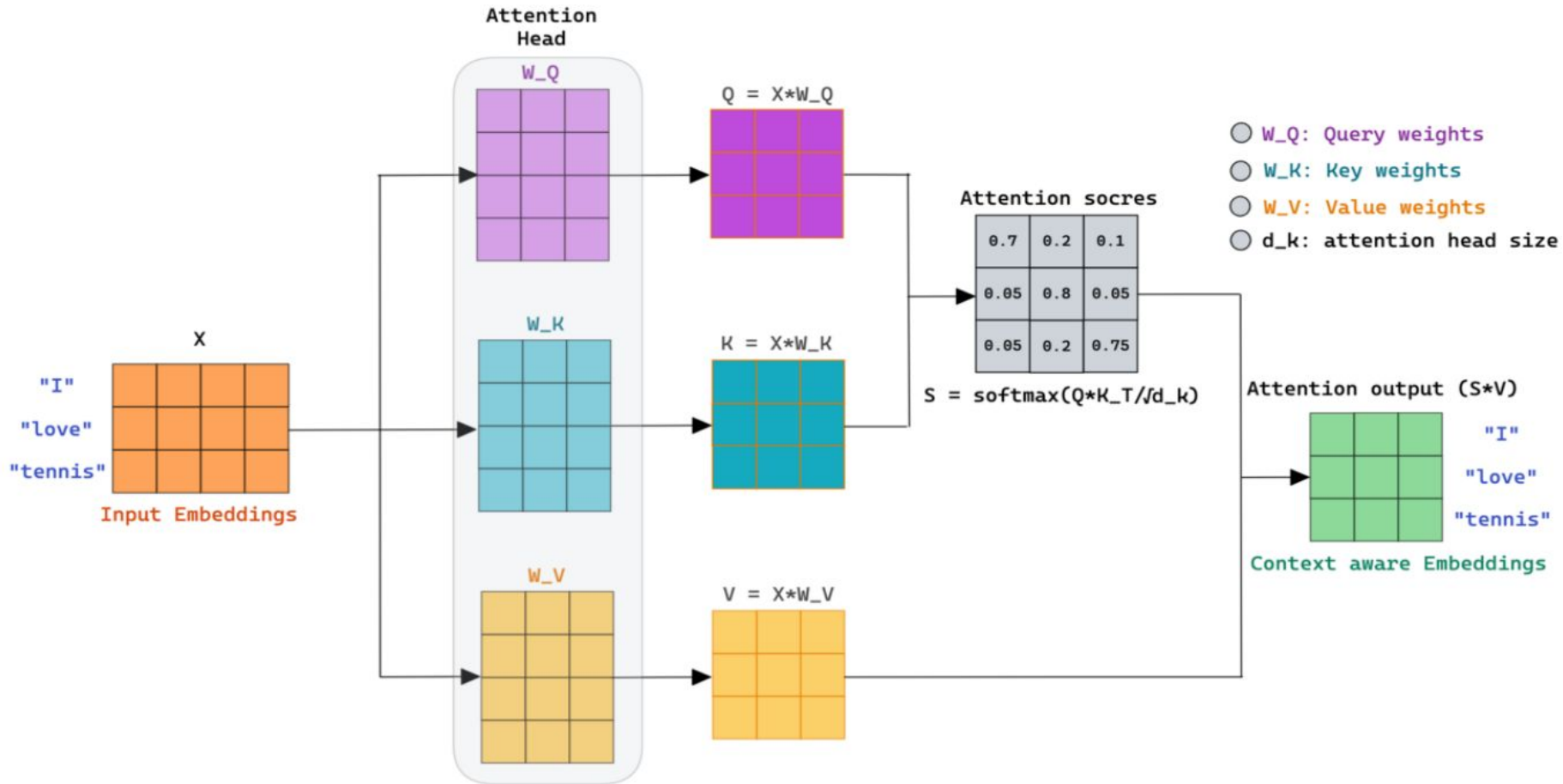
# Self Attention Mechanism:

Self-attention, also known as **scaled dot-product attention**, allows a model to weigh the importance of different tokens in a sequence relative to one another.

Each token in the sequence is compared to every other token, and attention scores are computed to determine how much focus each token should receive.

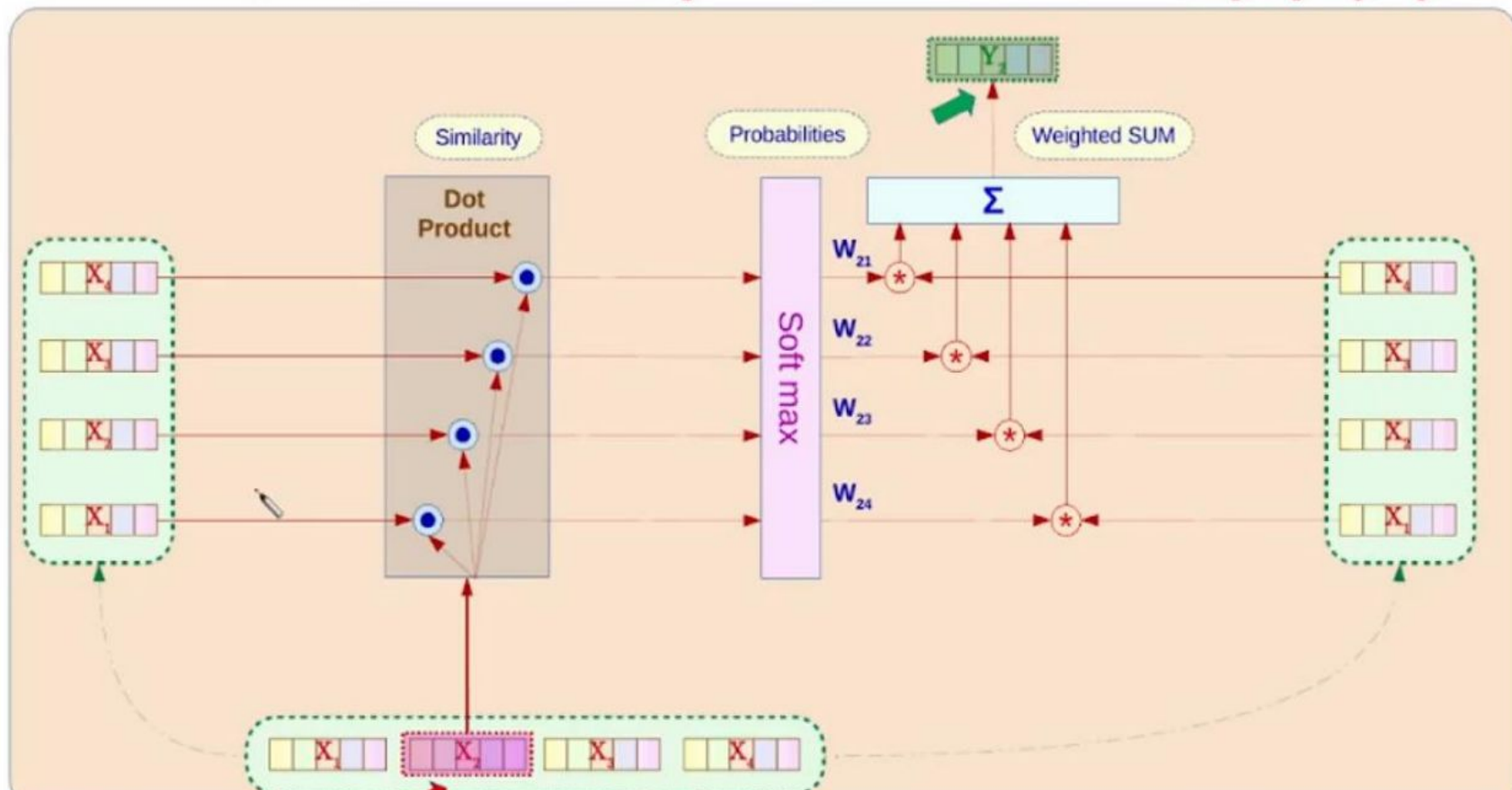


# Self-attention



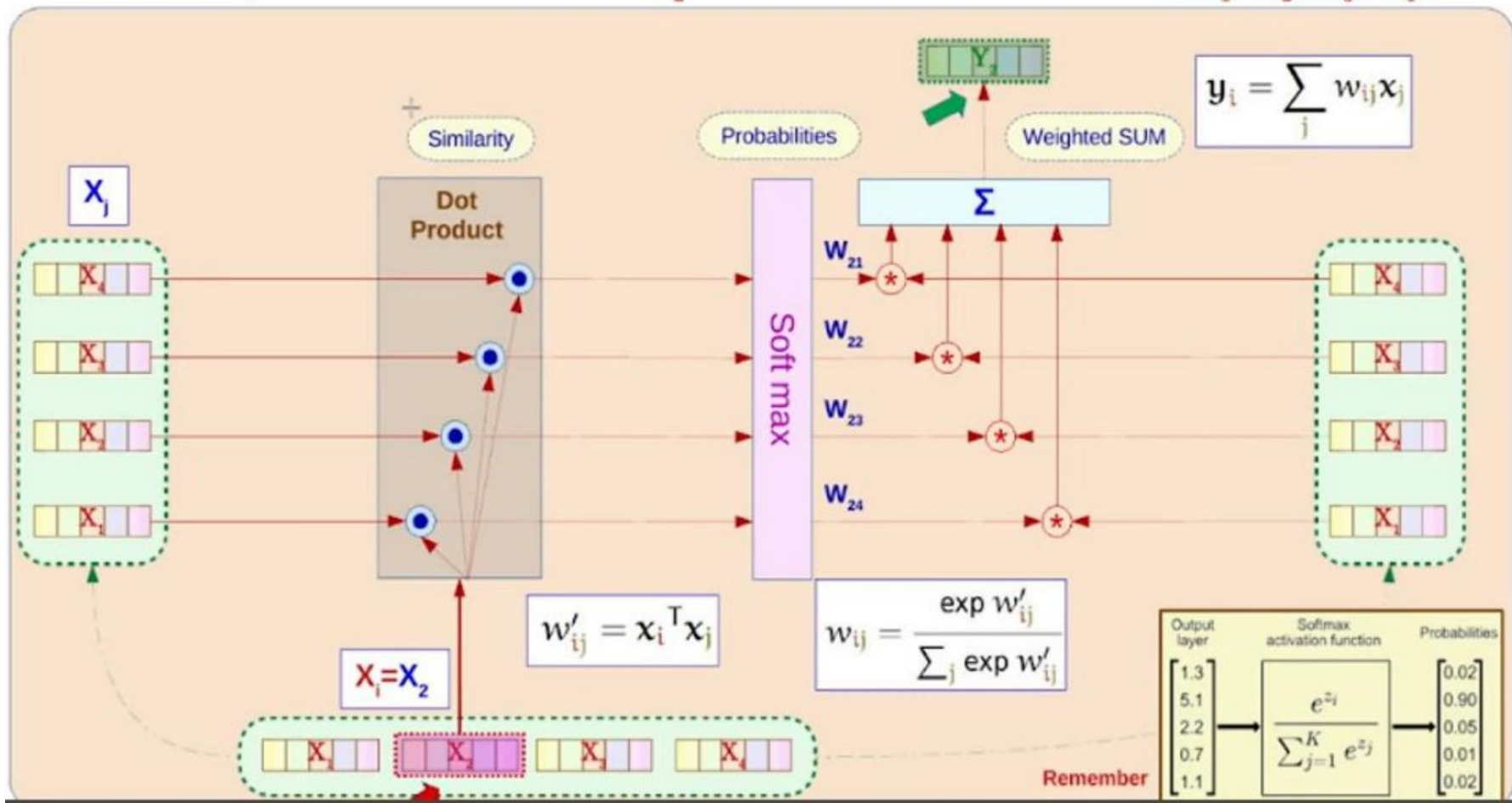
## Self Attention Mechanism (Fundamental Operation)

(Self Attention of word  $X_2$  w.r.t All words in Sentence ( $X_1, X_2, X_3, X_4$ ))



# Self Attention Mechanism (Fundamental Operation)

(Self Attention of word  $X_2$  w.r.t All words in Sentence ( $X_1, X_2, X_3, X_4$ ))



1- Each word is turned into 3 vectors:

a Query, a Key, and a Value (using learned weight matrices).

For each word (say “cat”),

2- we compare its Query ( $Q_2$ ) with all Keys ( $K_1, K_2, K_3, K_4$ ).

→ this gives attention scores = “how much should ‘cat’ look at each word?”

3-Apply softmax to make those scores into probabilities  
(so they sum to 1).

Example for “cat”:

maybe 0.1 to “The”, 0.6 to “sat”, 0.2 to “mat”, 0.1 to itself.

4- Multiply each Value (V) by these weights

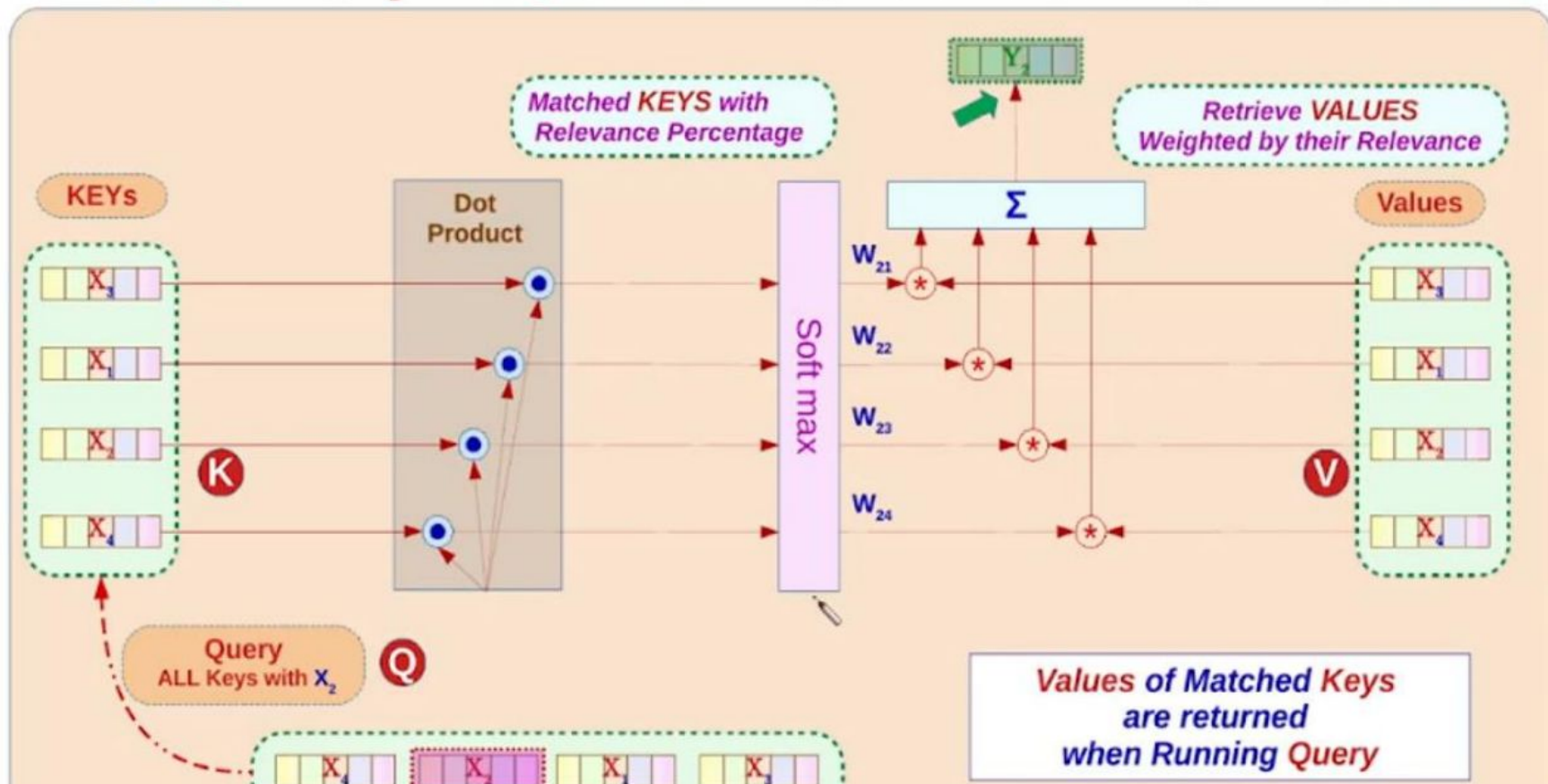
and sum them up →

this gives a new output vector for “cat” that contains info from all words.

5- Repeat for each word →

so every word's output now contains context from others.

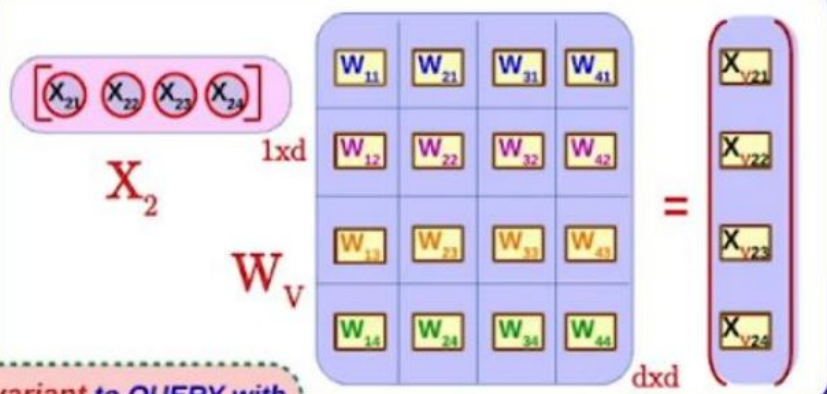
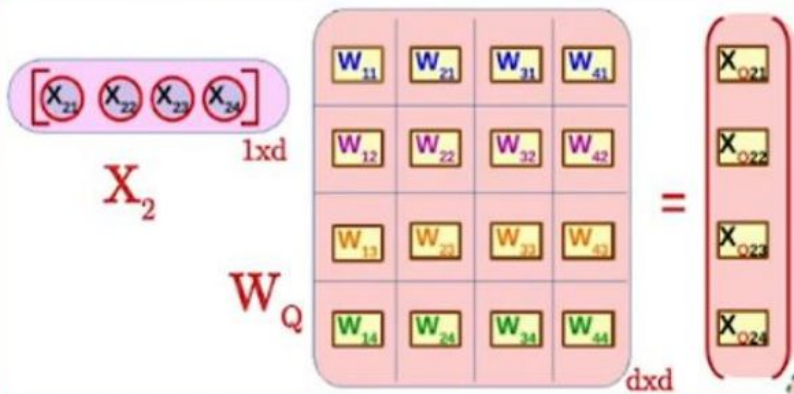
Self Attention Mechanism (as Query and {Key-Value} Dictionary)  
(Query using  $X_2$  All Keys in Dictionary and retrieve Corresponding Values)





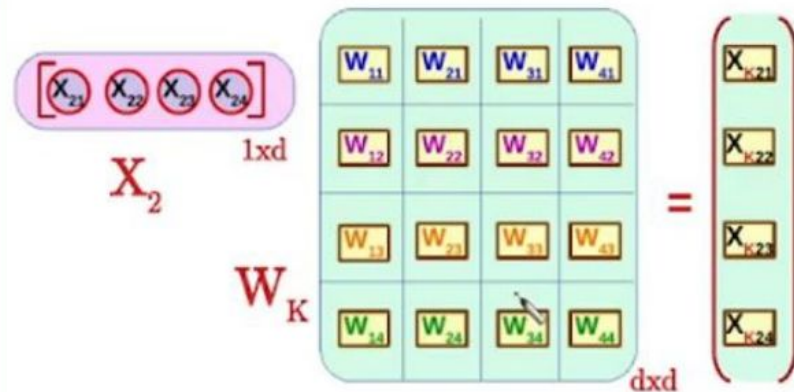
# Self Attention Mechanism (Linear Transformation of Inputs)

(Query using  $X_2$  All Keys in Dictionary and retrieve Corresponding Values)



$X_2$  variant to QUERY with

$X_2$  variant used as Value



$X_2$  variant used as KEY

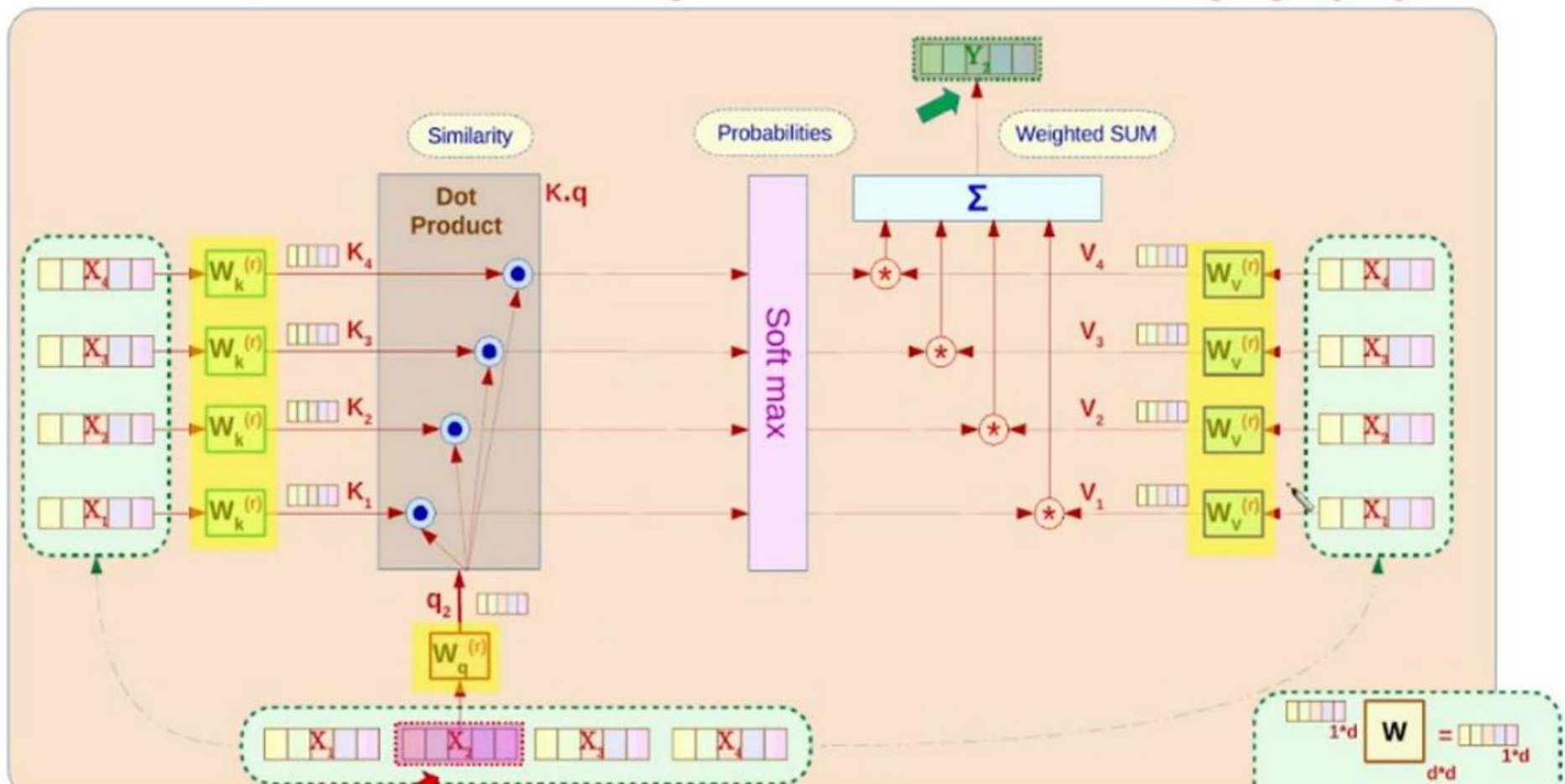
**Generate Three Variants of  $X_2$  (Based on its Role)**

$W_Q, W_K, W_V$  are controllable parameters (Trainable), allows to modify the incoming vectors to suit the three roles they must play



# Self Attention Mechanism (Linear Transformation of Inputs)

(Self Attention of word  $X_2$  w.r.t All words in Sentence ( $X_1, X_2, X_3, X_4$ ))



## **embedding dimension (d\_model) and softmax sensitivity:**

- It's the size (number of features) in each word's embedding.
- Example:  
If "cat"  $\rightarrow [0.2, 0.5, 0.1]$ , the dimension = 3.  
In real Transformers, d\_model = 512 or 768.

So each word is represented by a vector of that length.

How this affects attention scores?

In self-attention, we compute:

$$\text{scores} = Q \cdot K^T$$

Each score is the dot product between Query and Key vectors.

If the vectors have many dimensions (large  $d_{\text{model}}$ ),  
then the dot product will get larger, because you're summing over more numbers.

Example:

- If vectors are 3-D  $\rightarrow$  dot product  $\approx$  small (1–5)
- If vectors are 512-D  $\rightarrow$  dot product can be very large (maybe 50–100 or more)

## Softmax sensitivity:

Example      *Normal range*

$x = [2, 1, 0]$

$$e^x = [7.4, 2.7, 1]$$

Softmax = [0.66, 0.24, 0.09]

The model spreads attention somewhat — it's **balanced**.

---

Example      *Large values (scaled up)*

$x = [20, 10, 0]$

$$e^x = [4.85 \times 10^8, 2.2 \times 10^4, 1]$$

Softmax = [0.9999, 0.00004, 0.0000002]

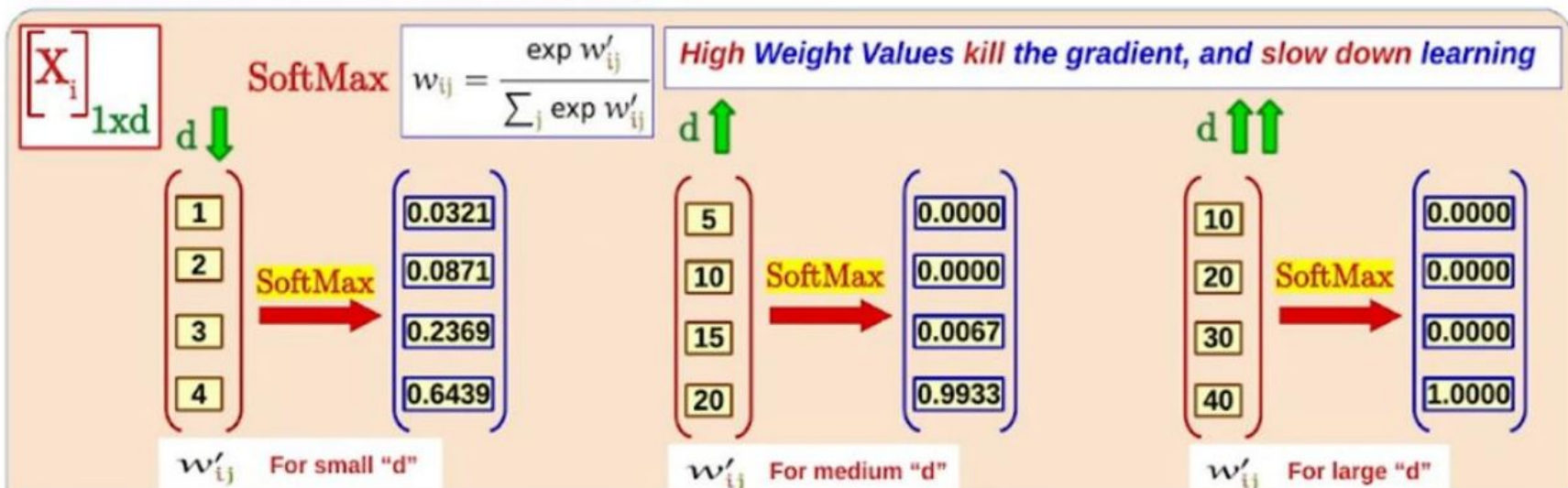
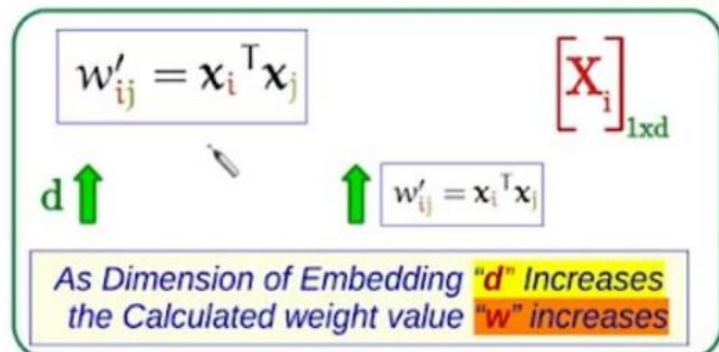
Now one value **completely dominates** — Softmax output is almost **one-hot**.  
The model **focuses on one word and ignores the rest**.

That's what we mean by **Softmax is sensitive**:

**Small differences in input** → **Huge differences in output**.

## Self Attention Mechanism

### (Embedding Vector Dimension and Softmax Sensitivity)



# Self Attention Mechanism

## (Scaling based on Square Root of Embedding Vector Dimension)

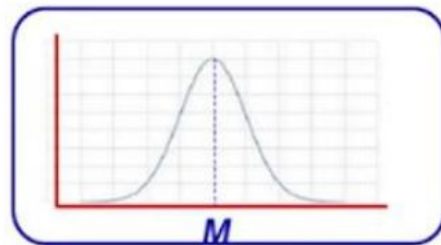
$$\begin{array}{c}
 V_1 \quad V_2 \\
 \begin{bmatrix} X_1 \\ X_2 \\ X_3 \\ X_4 \\ X_5 \\ X_6 \\ X_7 \\ X_8 \\ X_9 \\ X_{10} \\ X_{11} \\ X_{12} \end{bmatrix} \quad \begin{bmatrix} X_1 \\ X_2 \\ X_3 \\ X_4 \end{bmatrix} \\
 1 \times d_1 \quad 1 \times d_2
 \end{array}$$

Average Values of elements of  $V_1 = M$

Average Values of elements of  $V_2 = M$

$$\text{Vector Length of } V_1 \text{ (all Ms)} = V_1^T V_1 = M\sqrt{d_1}$$

$$\text{Vector Length of } V_2 \text{ (all Ms)} = V_2^T V_2 = M\sqrt{d_2}$$



$\sqrt{d}$  is a good choice for scaling

$\sqrt[4]{d}$  Some times is used

**Remember:**

"d" is  
Dimension of Embedding

Not  
length of input Sequence

$$\text{SoftMax} \left[ w'_{ij} \right]$$

$$\text{SoftMax} \left[ \frac{w'_{ij}}{\sqrt{d}} \right]$$

$$d=100, \sqrt{d}=10$$

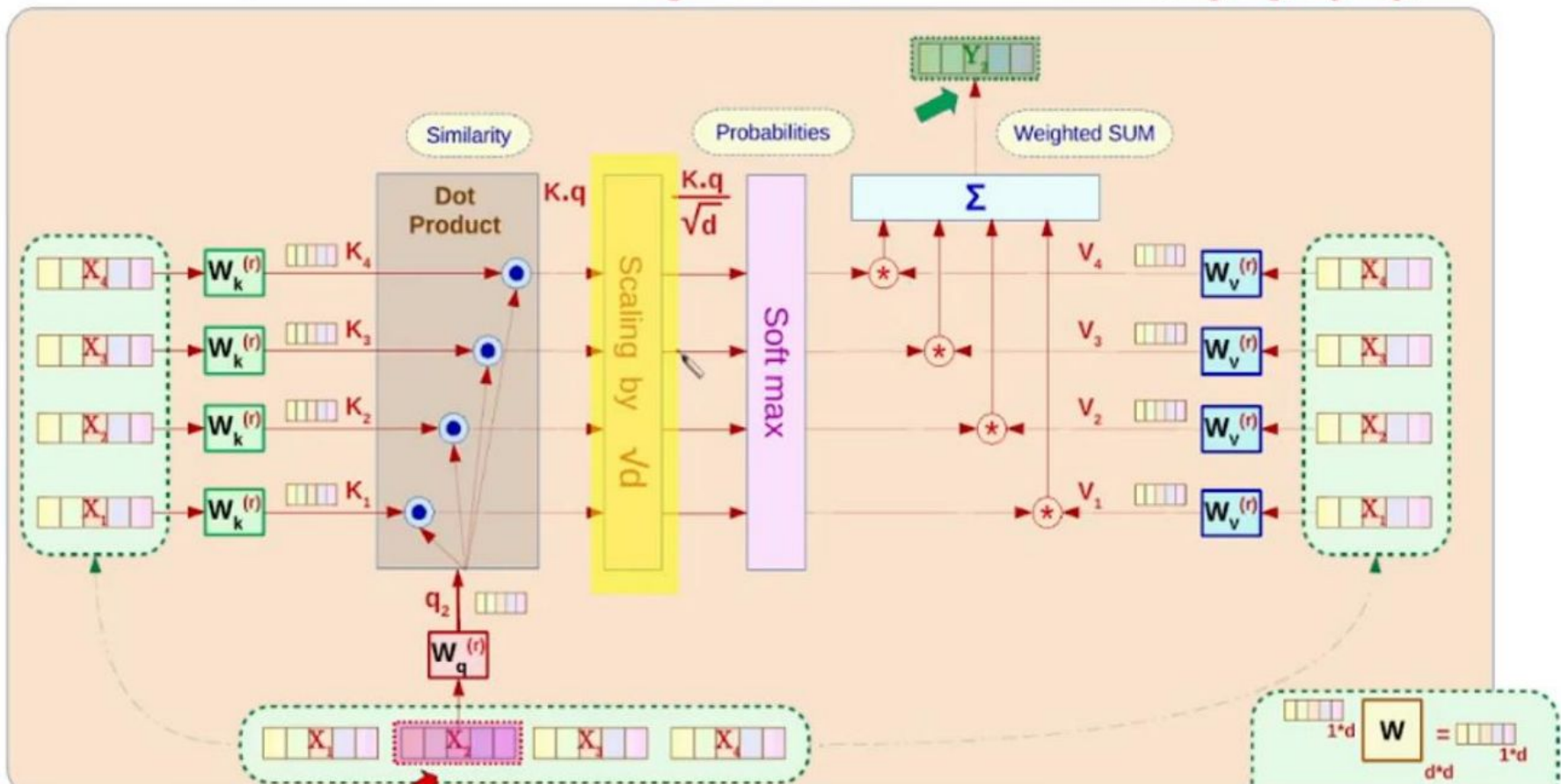
$$\begin{array}{c}
 \begin{bmatrix} 10 \\ 20 \\ 30 \\ 40 \end{bmatrix} \xrightarrow{\text{SoftMax}} \begin{bmatrix} 0.0000 \\ 0.0000 \\ 0.0000 \\ 1.0000 \end{bmatrix}
 \end{array}$$

$$\begin{array}{c}
 \begin{bmatrix} 10 \\ 20 \\ 30 \\ 40 \end{bmatrix} / \sqrt{d} \xrightarrow{\text{Scaled}} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \xrightarrow{\text{SoftMax}} \begin{bmatrix} 0.0321 \\ 0.0871 \\ 0.2369 \\ 0.6439 \end{bmatrix}
 \end{array}$$



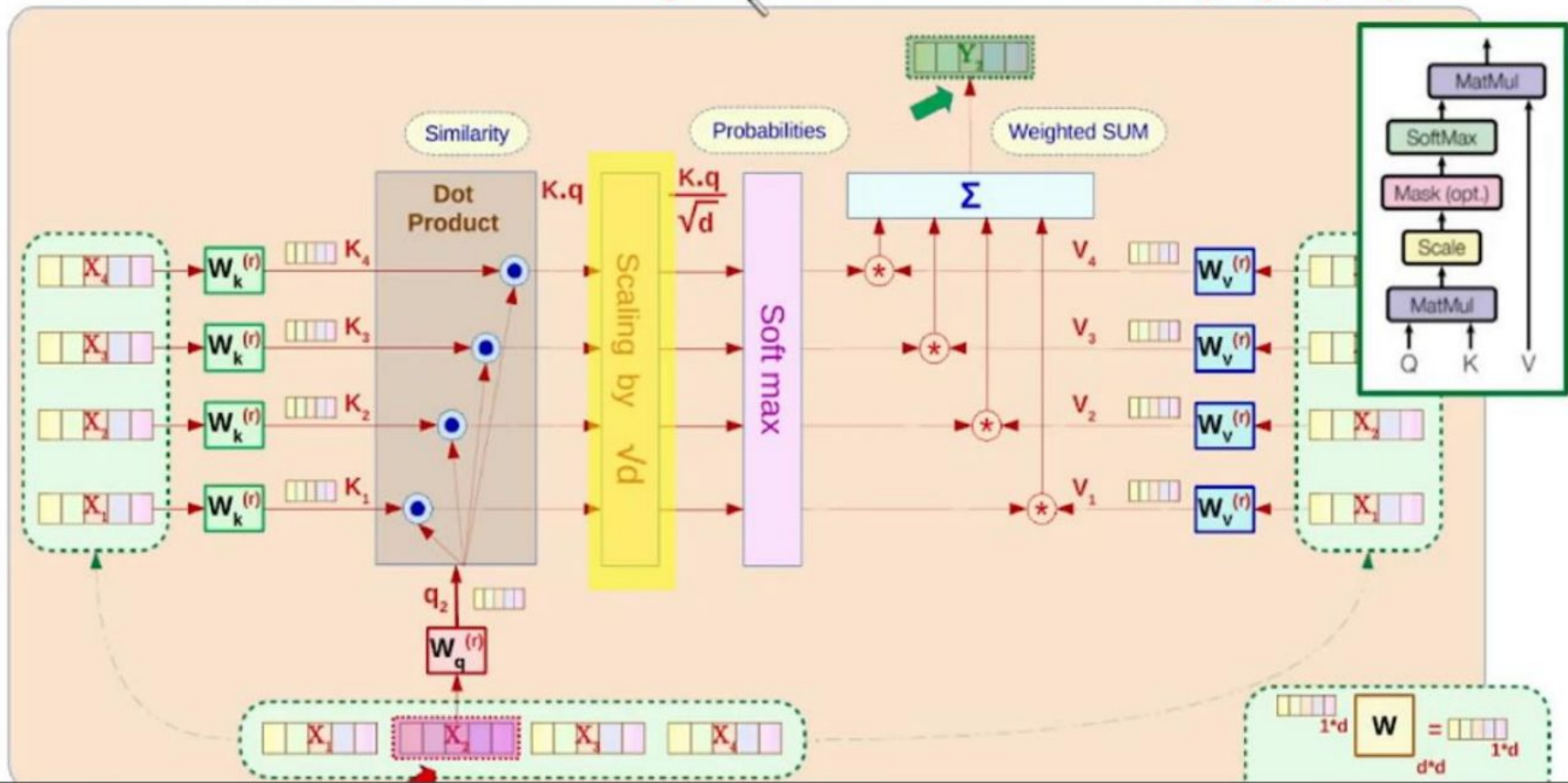
# Self Attention Mechanism (Adding Scaling)

(Self Attention of word  $X_2$  w.r.t All words in Sentence ( $X_1, X_2, X_3, X_4$ ))



# Self Attention Mechanism (Adding Scaling)

(Self Attention of word  $X_2$  w.r.t All words in Sentence ( $X_1, X_2, X_3, X_4$ ))





In self-attention, we compute:

$$\text{Attention weights} = \text{Softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right)$$

If  $QK^T$  values are large, softmax will give almost all attention to **one token**, making the model:

- Too confident
- Less able to mix context from other words
- Harder to train, because gradients vanish for ignored words

This is why we call it softmax sensitivity.

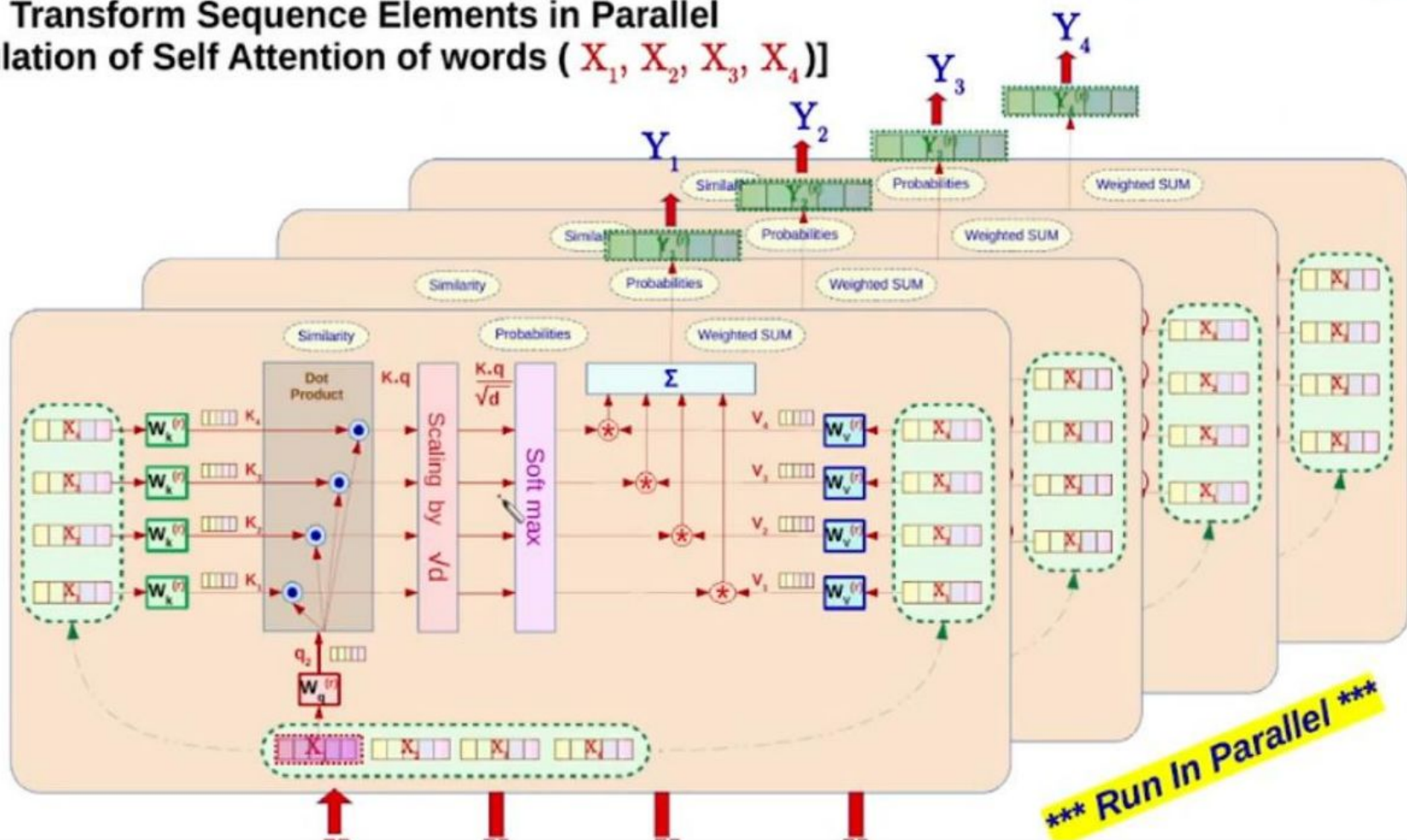
## The fix: Scaling

We divide by  $\sqrt{d_k}$  to keep the dot-product values small and stable.

# Parallel Processing

## Transform Sequence Elements in Parallel

[Calculation of Self Attention of words ( $X_1, X_2, X_3, X_4$ )]



# Multi-Head Attention



**Wide Architecture**

**Narrow Architecture**



# Multi-Head Self Attention

Combining several self attention mechanisms give the self attention greater power of discrimination

Each Head is indexed with “ $r$ ”

Each head has its own matrices  $W_q^r$ ,  $W_k^r$ ,  $W_v^r$ .

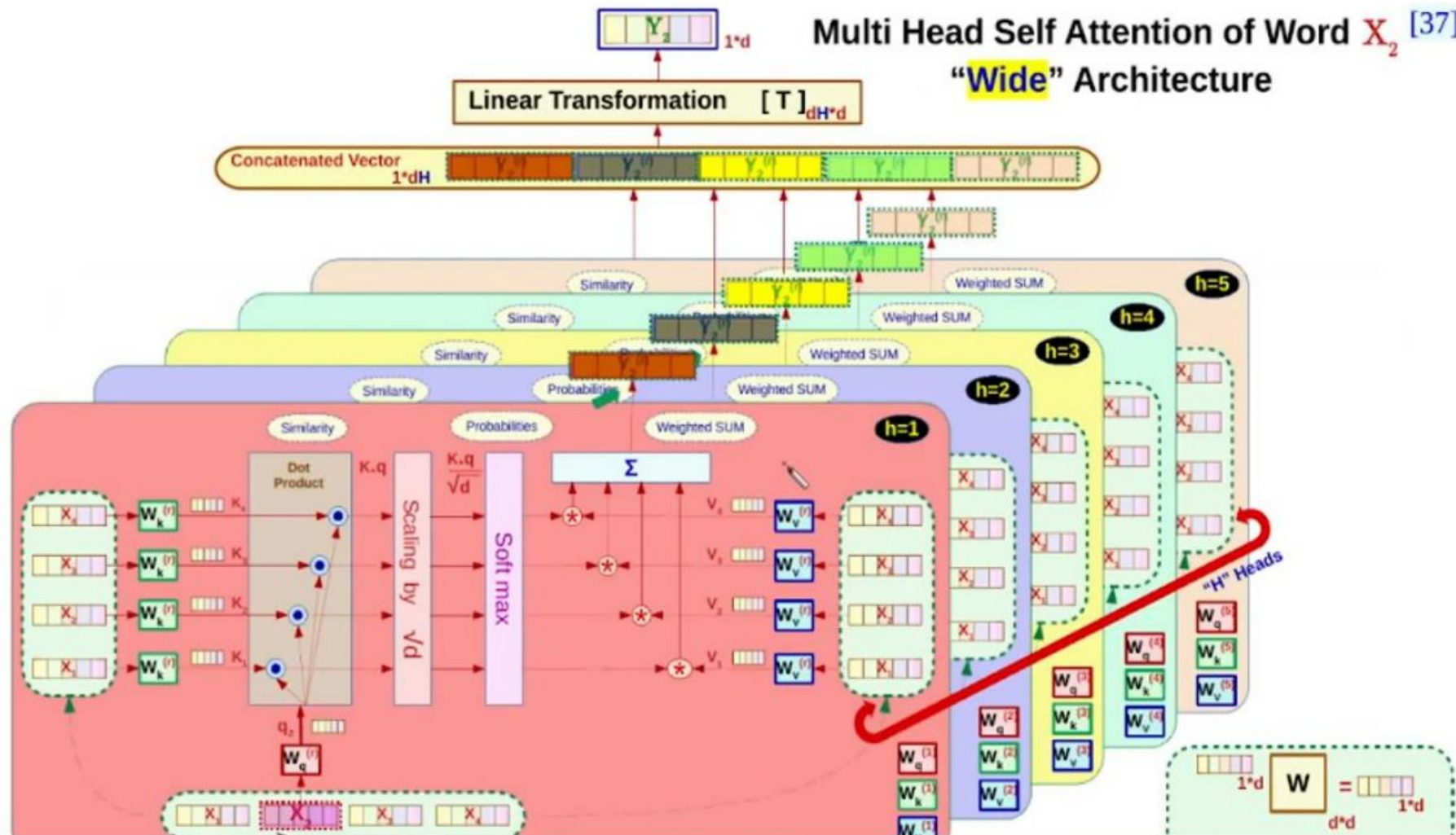
For input  $X$  each attention head produces a different output vector  $Y^r$ .

All Output vectors are concatenated, and pass through a **linear transformation** to reduce the dimension back to original  $X$  dimension.



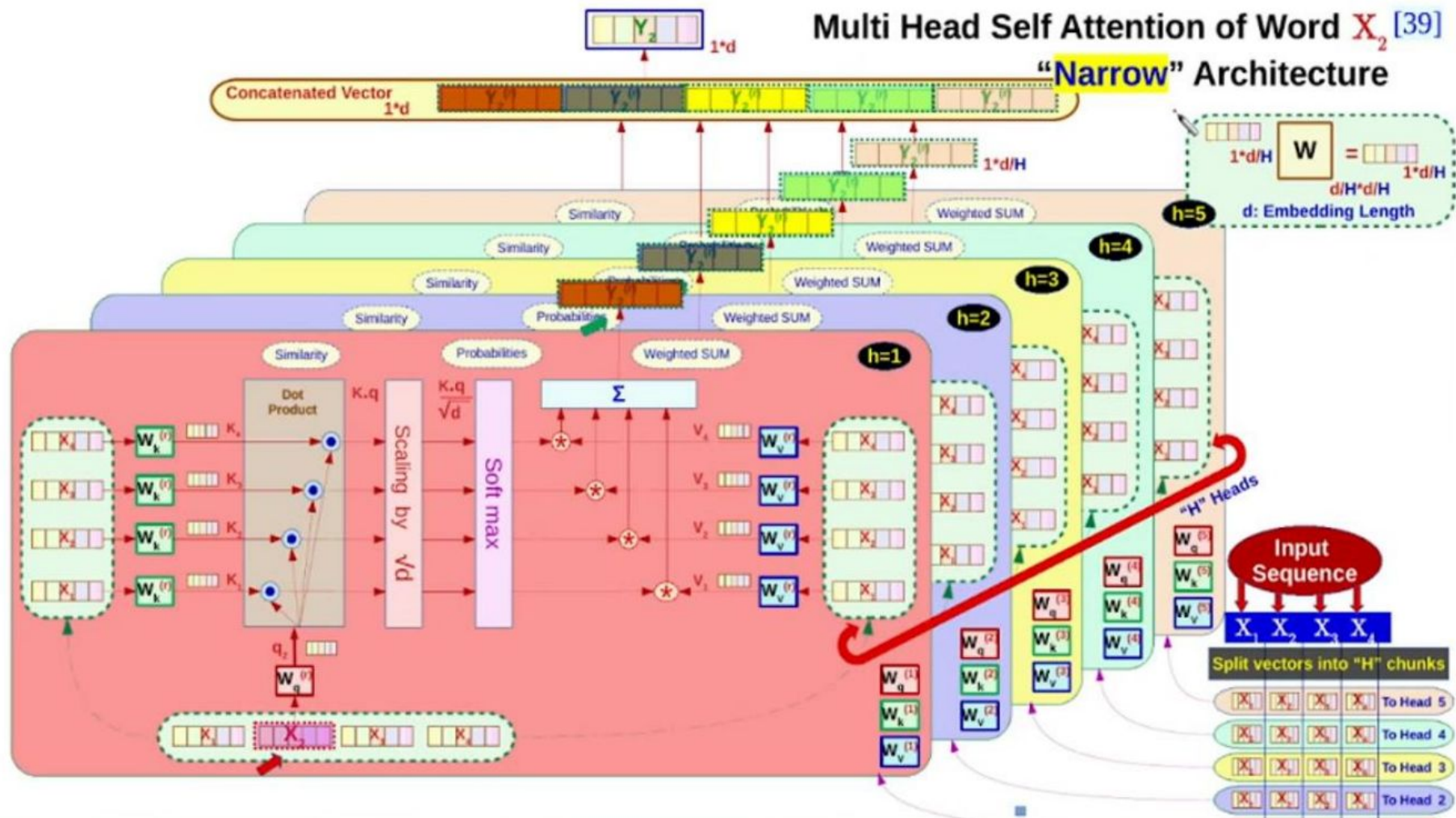
# Multi Head Self Attention of Word $X_2$ [37]

## "Wide" Architecture



# Multi Head Self Attention of Word $X_2$ [39]

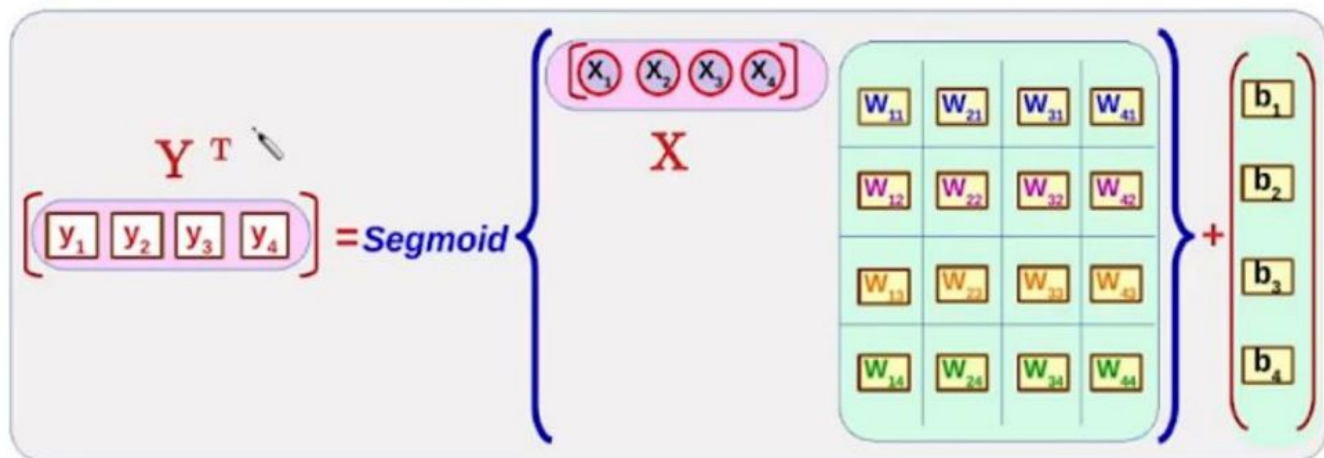
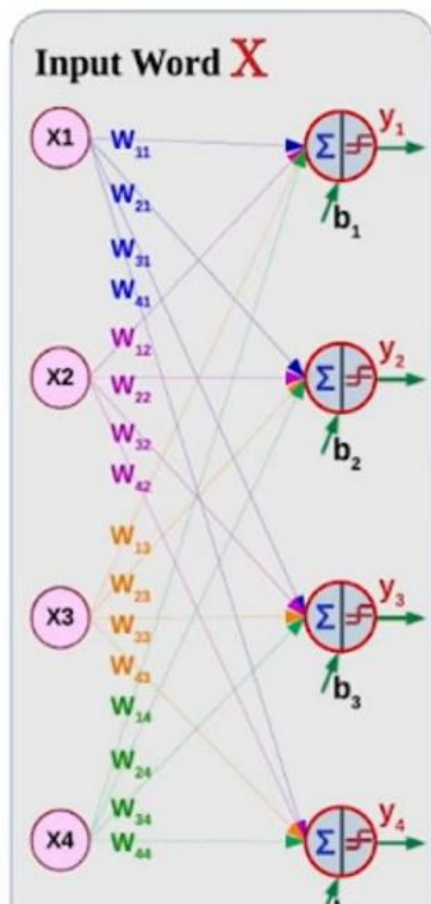
“Narrow” Architecture



# Parallelization Implementation Issues



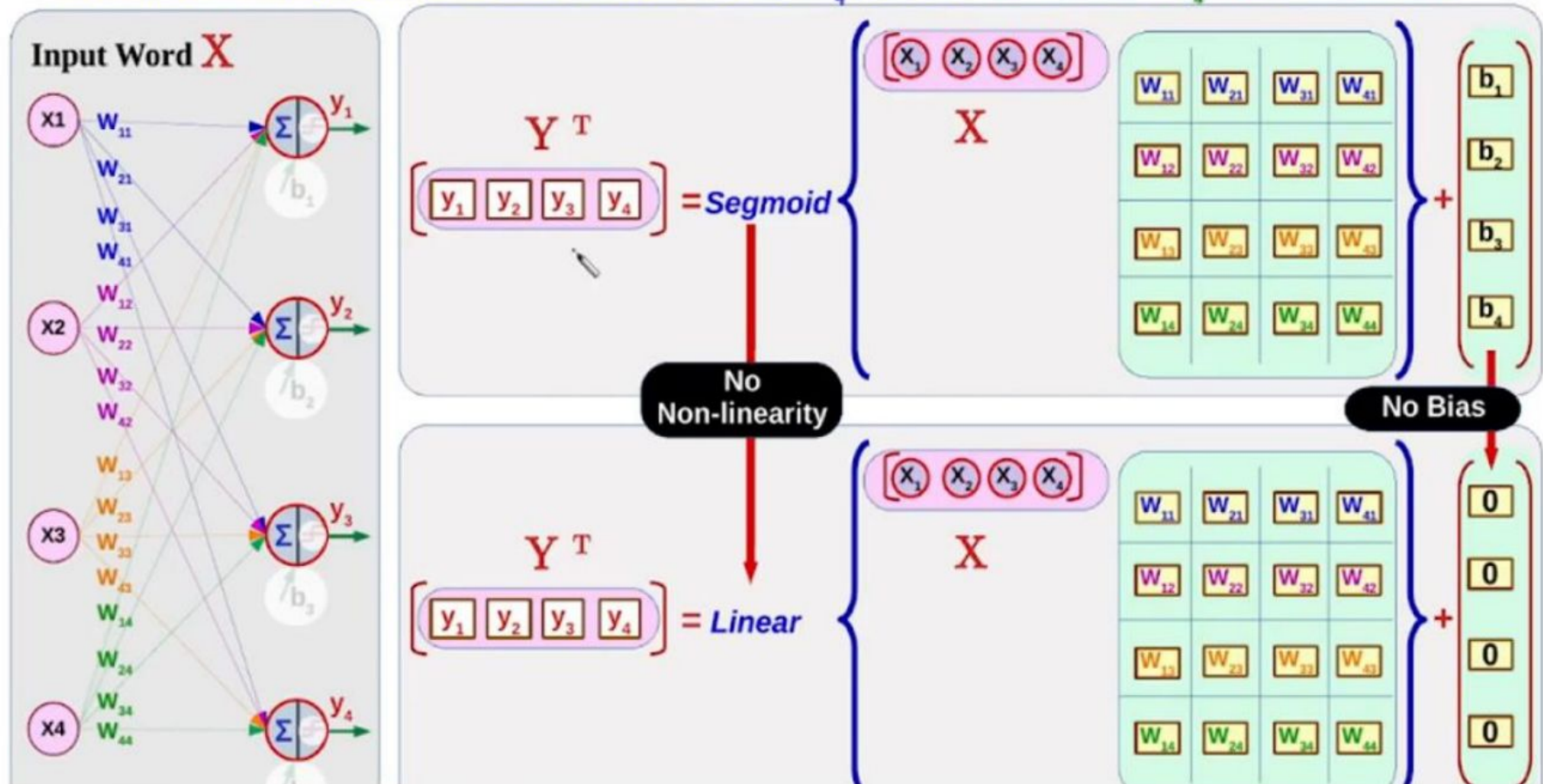
# Using FF NN for Multiplication of a Vector by a Matrix



## Using FF NN for Multiplication of ONE Vector by a Matrix

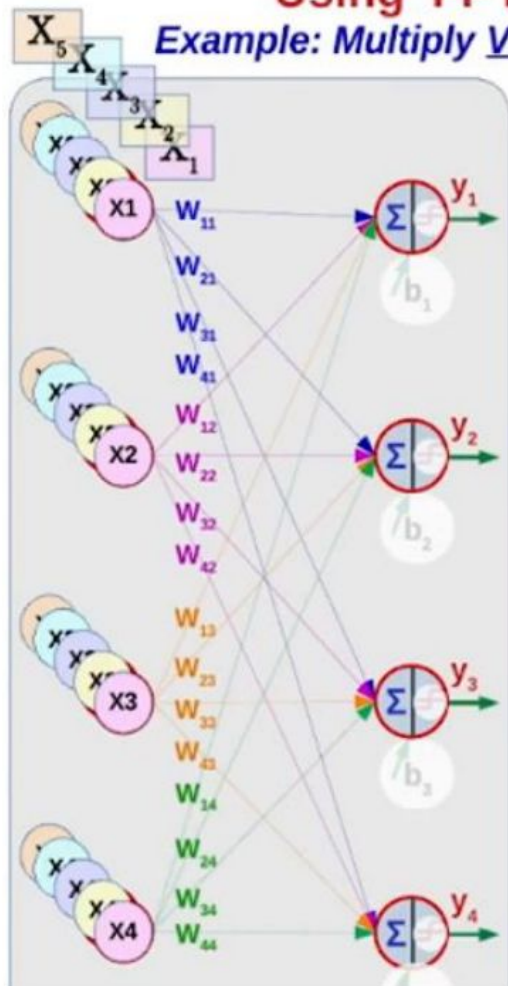
[42]

Example: Multiply Vector of Current Word  $X$  by  $W_q$  (Query) {Remember:  $W_q$  is Squared Matrix}



# Using FF NN for Multiplication of Multiple Vectors by a Matrix

Example: Multiply Vectors of ALL Words by  $W_k$  (Key) or  $W_v$  (Value)  $\{W_k, W_v \text{ is Squared Matrix}\}$

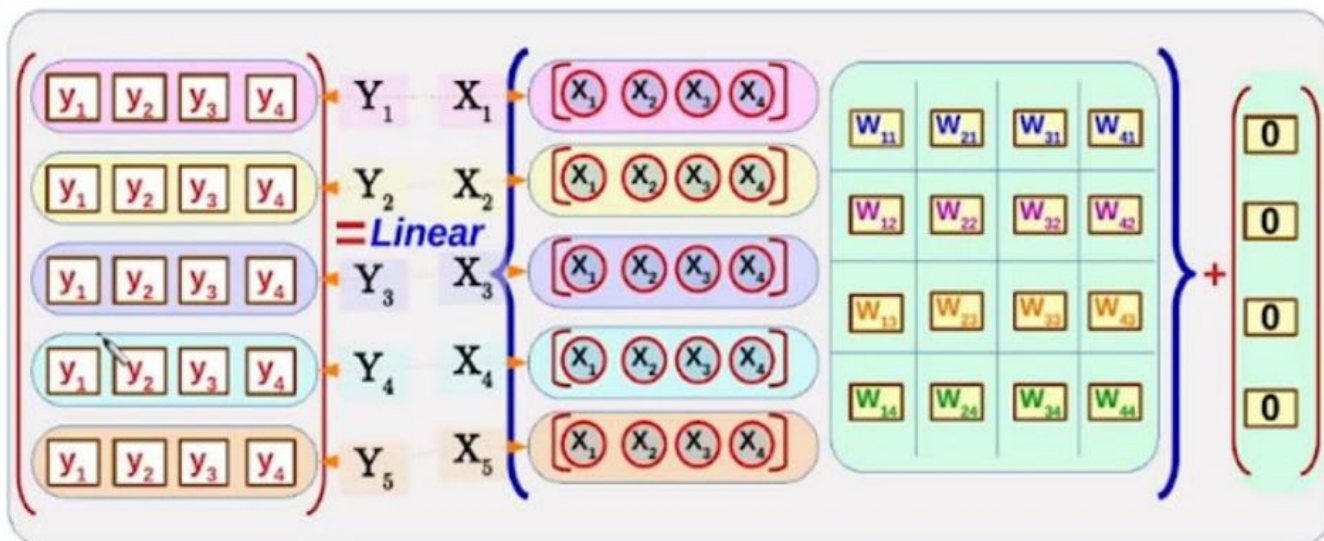


**Design FF NN with the following Considerations:**

**Activation:** Linear

**Bias:** Zeros

**Inputs:** All Words Within Same Paragraph ( $X_1, X_2, X_3, X_4, X_5$ )



# Residual Connection:

Residual Connections are a type of skip-connection that learn residual functions with reference to the layer inputs, instead of learning unreferenced functions.

Residual connection provides another path for data to reach latter parts of the neural network by skipping some layers.

Without residual connections, adding more layers to a neural network can often degrade performance due to optimization difficulties.

Residual connections make it easier to train very deep networks by allowing the model to learn identity mappings more easily.

This means that if adding more layers doesn't improve the performance, the residual connections can help the network learn to essentially "skip" those layers, preventing a degradation in performance.

## **Improving Generalization:**

By enabling the training of deeper networks, residual connections allow the model to potentially learn more complex features and patterns from the data. This can lead to better generalization on unseen data, improving the overall performance of the model.

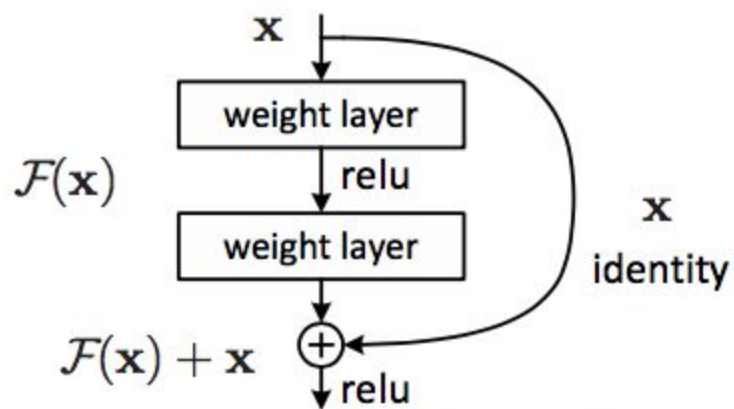
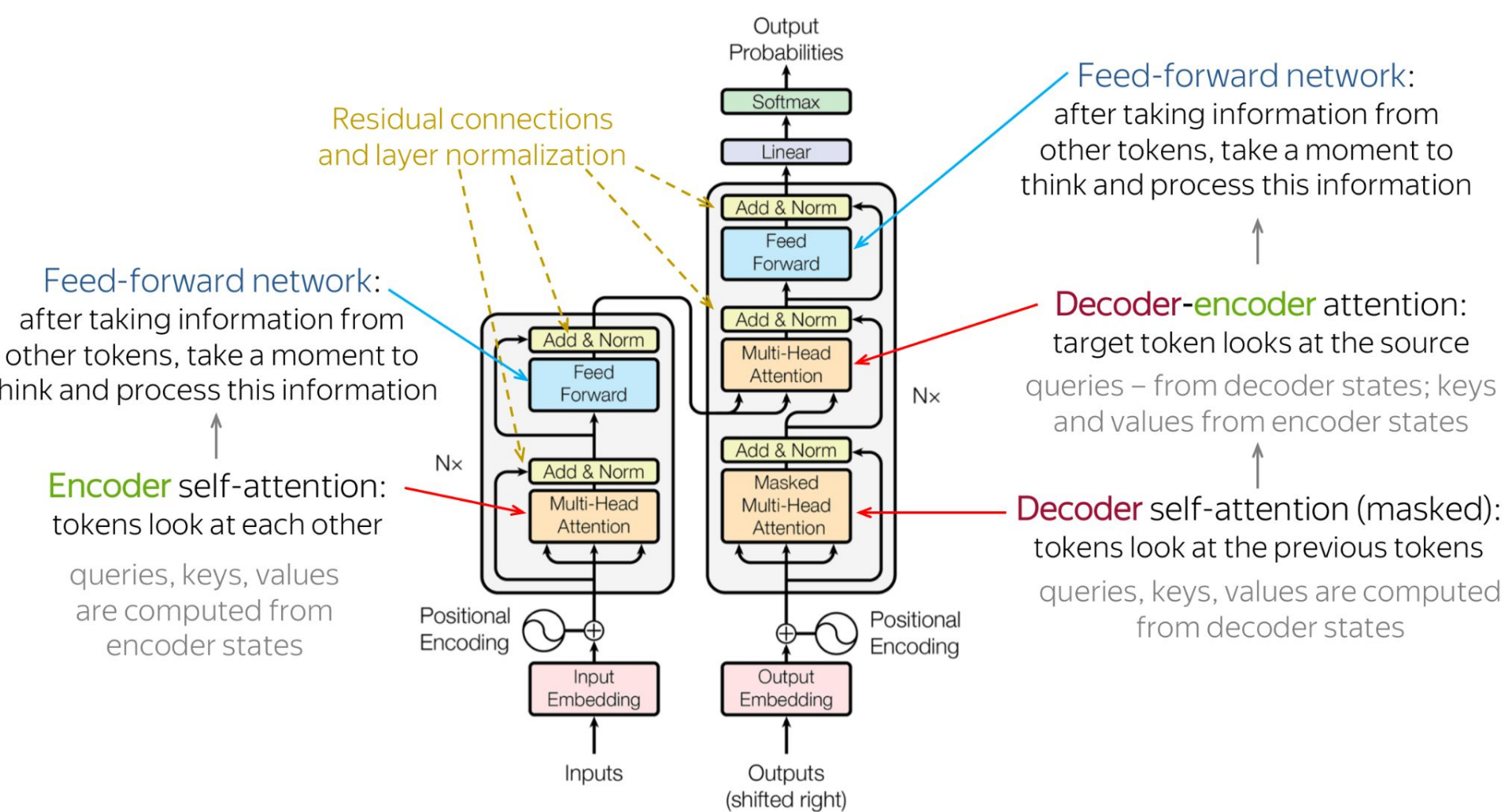


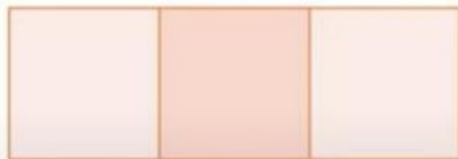
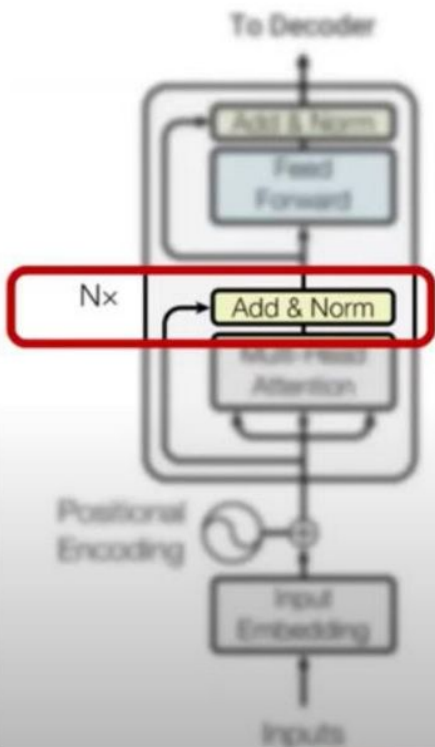
Figure 2. Residual learning: a building block.





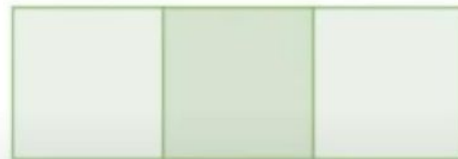
# Encoder

## Add and Normalization



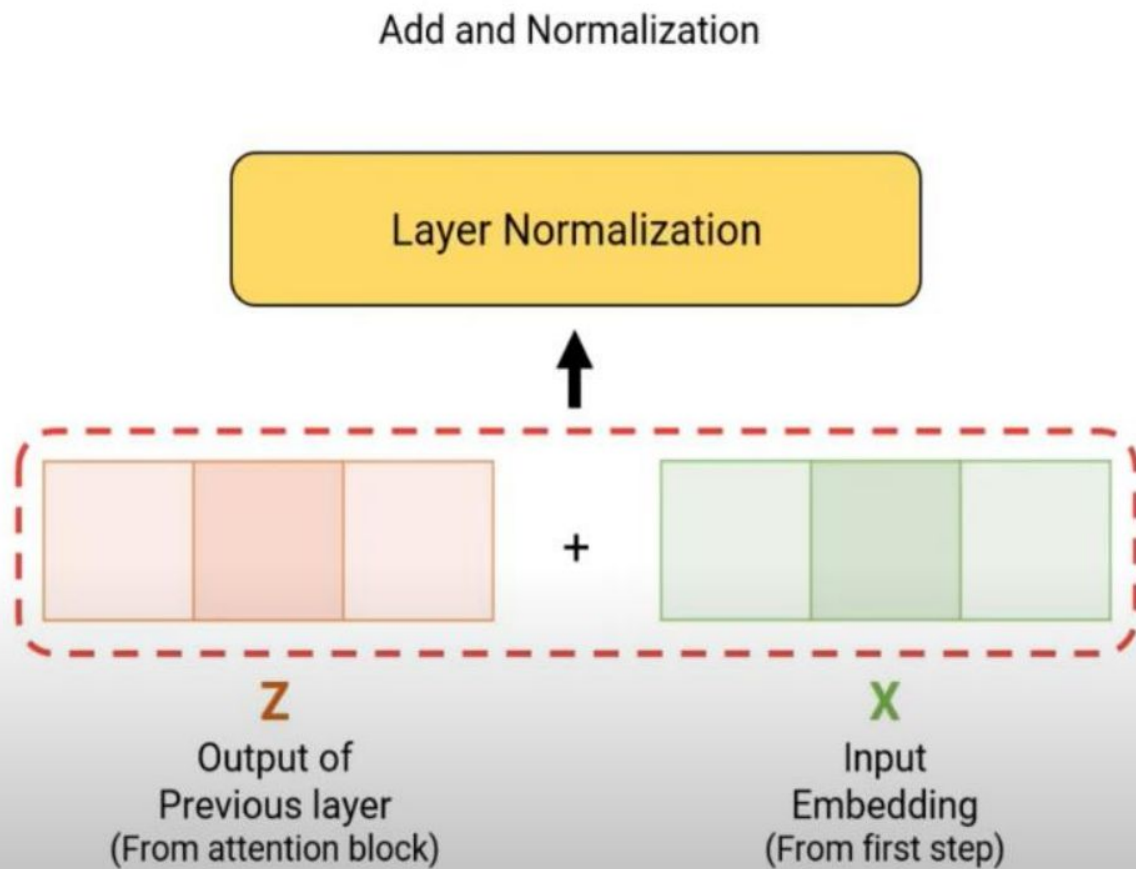
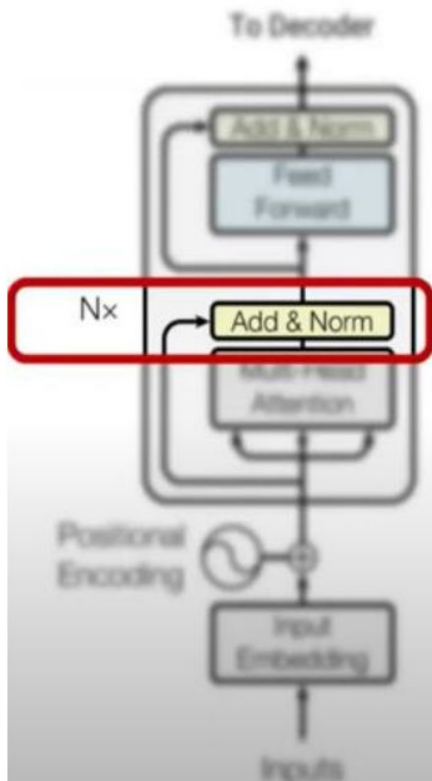
**Z**  
Output of  
Previous layer  
(From attention block)

+

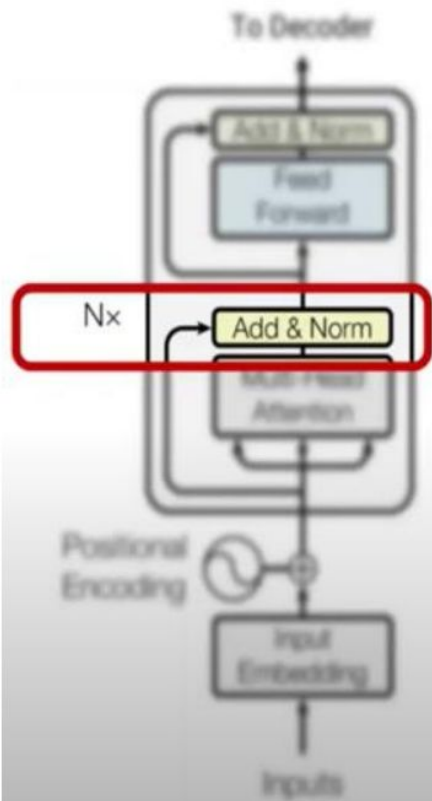


**X**  
Input  
Embedding  
(From first step)

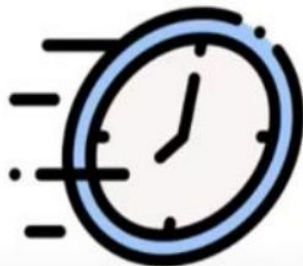
# Encoder



# Encoder



## Benefits of Normalization



Faster Training



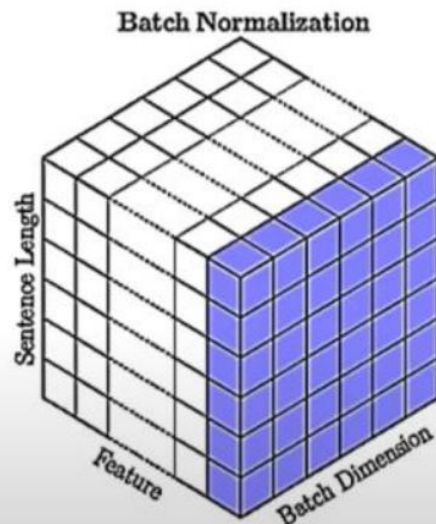
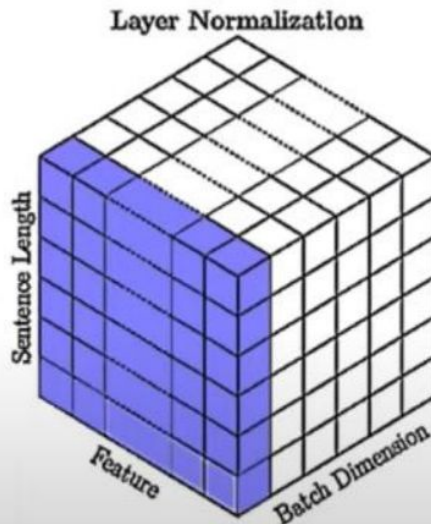
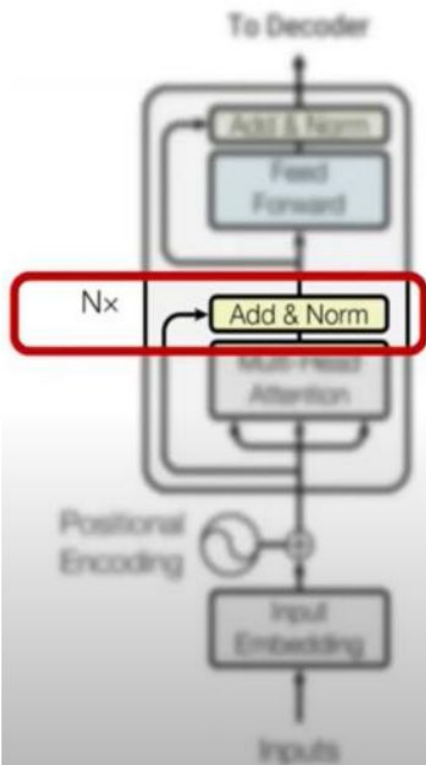
Reduce Bias



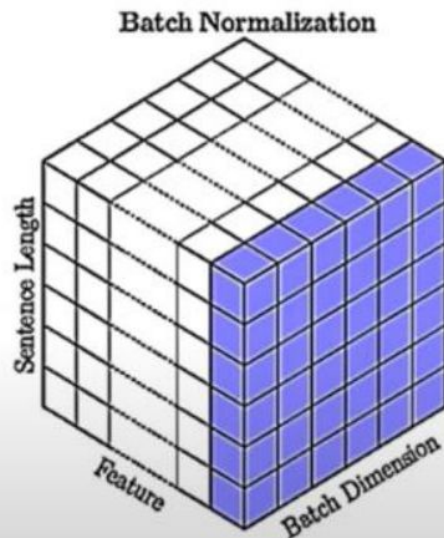
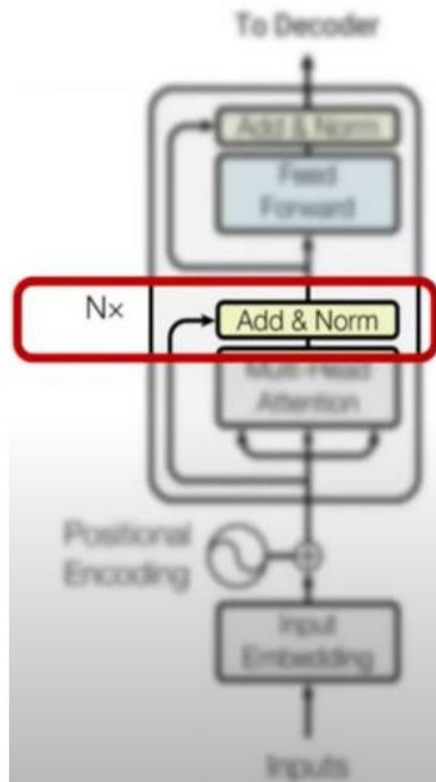
Prevent weight explosion

# Encoder

## Normalization techniques



# Encoder



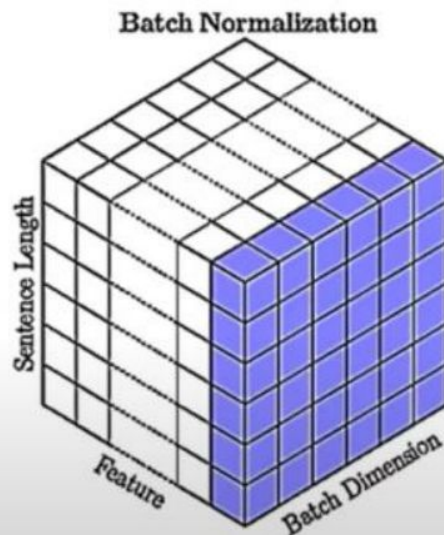
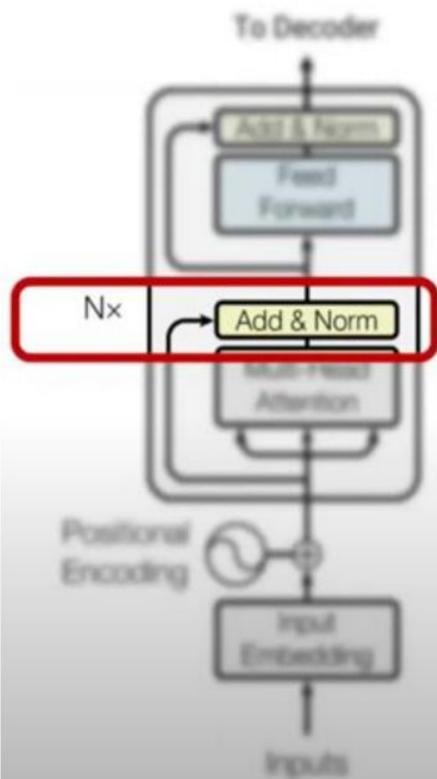
## Batch Normalization

|         |      |      |      |
|---------|------|------|------|
| Popcorn | 0.31 | 0.14 | 0.93 |
| Popped  | 0.14 | 0.88 | 0.98 |
| Tea     | 0.85 | 0.20 | 0.14 |
| Steeped | 0.46 | 0.61 | 0.49 |

- Average = 0.44
- Variance = 0.07



# Encoder



## Batch Normalization

Popcorn

|       |      |      |
|-------|------|------|
| -0.49 | 0.14 | 0.93 |
| -1.14 | 0.88 | 0.98 |

Popped

Tea

|       |      |      |
|-------|------|------|
| 1.56  | 0.20 | 0.14 |
| 0.076 | 0.61 | 0.49 |

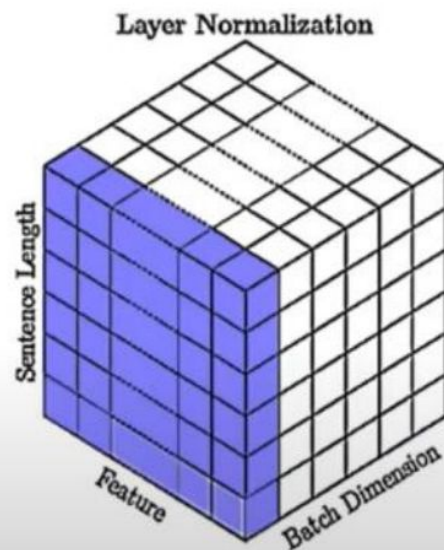
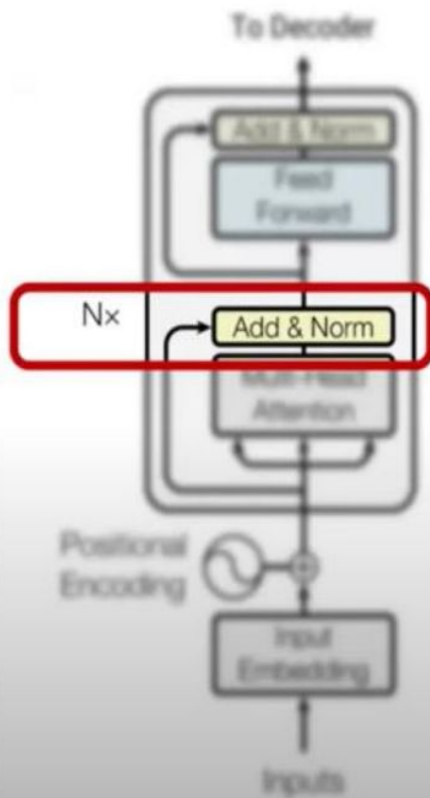
Steeped



Repeat this for other features



# Encoder



## Layer Normalization

Popcorn

|      |      |      |
|------|------|------|
| 0.31 | 0.14 | 0.93 |
| 0.14 | 0.88 | 0.98 |

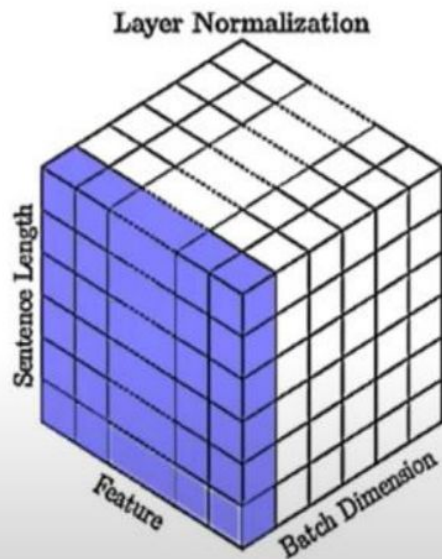
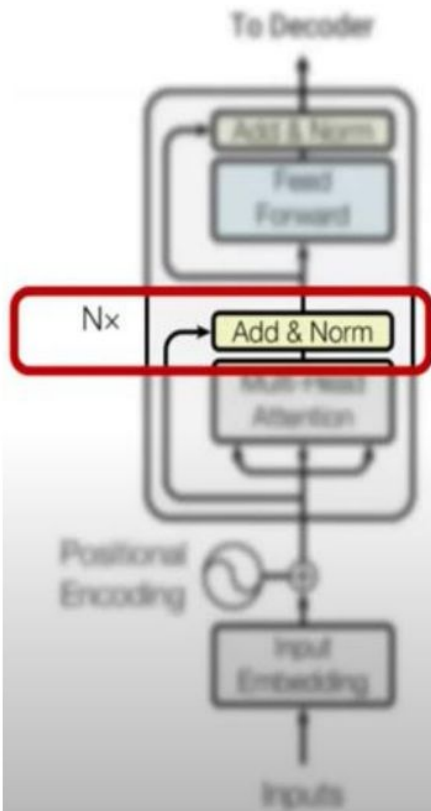
Popped

Tea

|      |      |      |
|------|------|------|
| 0.85 | 0.20 | 0.14 |
| 0.46 | 0.61 | 0.49 |

Steeped

# Encoder



## Layer Normalization

Popcorn

|      |      |      |
|------|------|------|
| 0.31 | 0.14 | 0.93 |
| 0.14 | 0.88 | 0.98 |

Popped

- Average = 0.56
- Variance = 0.09

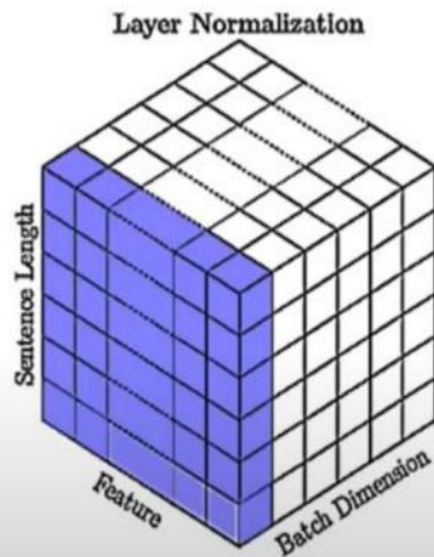
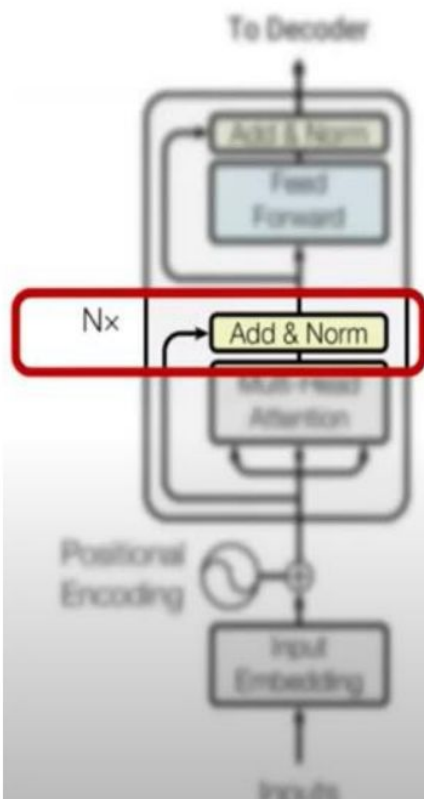
Tea

|      |      |      |
|------|------|------|
| 0.85 | 0.20 | 0.14 |
| 0.46 | 0.61 | 0.49 |

Steeped

- Average = 0.45
- Variance = 0.05

# Encoder



## Layer Normalization

Popcorn

|       |       |      |
|-------|-------|------|
| -0.83 | -1.38 | 1.20 |
| -1.38 | 1.04  | 1.37 |

- ✓ **Normalized!**
- Average = 0
  - Variance = 0.99

Popped

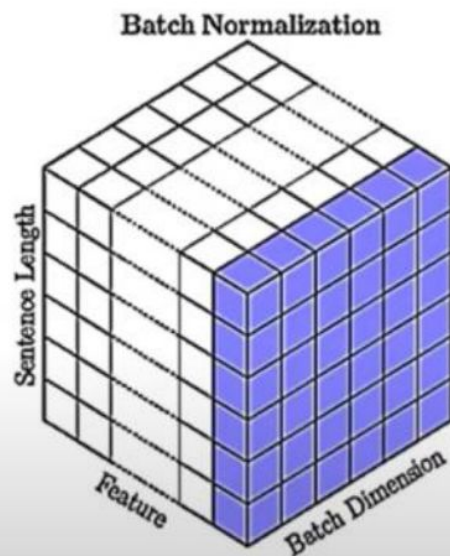
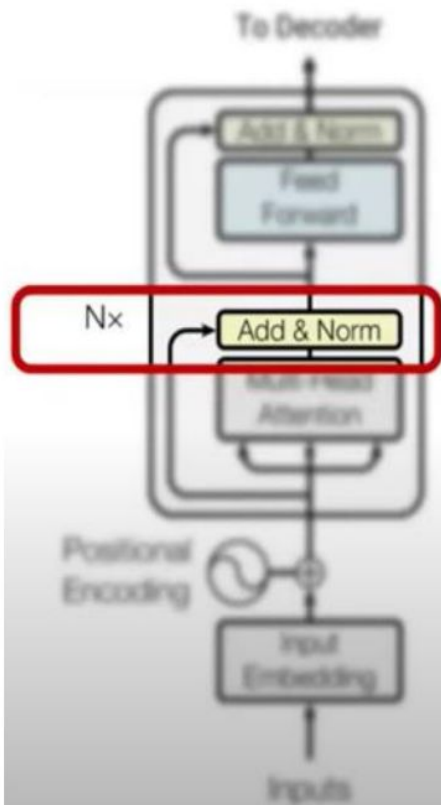
Tea

|      |       |       |
|------|-------|-------|
| 1.69 | -1.11 | -1.37 |
| 0.01 | 0.65  | 0.13  |

- ✓ **Normalized!**
- Average = 0
  - Variance = 0.99

Steeped

# Encoder



## Batch Normalization

A table showing normalized values for four items: Popcorn, Popped, Tea, and Steeped. The values are arranged in a 4x3 grid. The first two rows (Popcorn, Popped) are highlighted in blue, and the last two rows (Tea, Steeped) are highlighted in orange. A green dashed box encloses the entire table, with a green arrow pointing down towards the summary text.

|         |       |      |      |
|---------|-------|------|------|
| Popcorn | -0.49 | 0.14 | 0.93 |
| Popped  | -1.14 | 0.88 | 0.98 |
| Tea     | 1.56  | 0.20 | 0.14 |
| Steeped | 0.076 | 0.61 | 0.49 |

✓ **Normalized!**

- Average = 0
- Variance = 0.99